

休闲零食连锁店商品销量预测

小组每位成员信息：

学号	姓名	性别	学院	班级	电话
25011076	吴渔桐	男	化学与分子工程学 院	化学类 (新工 科)2503	18101855669
25012721	张哲凌	男	数学学院	2503	15026542486
25010409	朱饶杰	男	材料科学与工程学 院	2514 班	18101609530

2026 年 5 月 31 日

摘要

本文针对休闲零食连锁店商品销量预测问题，基于历史零售明细和天气、节假日、促销活动数据，构造日期-门店-商品粒度的日销量面板。数据清洗阶段确认销售数据覆盖 2020-04-01 至 2022-03-31，包含 7 家门店、12 个商品代码和 8 个商品类别；负销量按人工确认为损耗或冲销类负向调整，不作为正常购买需求，预测目标采用正向销量。

针对四个问题，本文采用可解释、可复现的分层建模路线。问题一使用移动平均、同星期均值和简单指数平滑分析门店、商品和门店-商品组合销量，门店层面以 7 日移动平均为主，商品和门店-商品层面以简单指数平滑为主，并补充 7 日窗口总量验证。问题二以附件类别字段作为同类零食的正式聚合依据，使用 Pearson、Spearman、去星期效应相关和门店内稳定性检查分析商品同步波动，类别预测采用“类别聚合后直接预测 + 简单指数平滑”。问题三以“合并天气 + 对数销量 + 历史控制固定效应回归”为主分析，描述性统计、温度分箱、去异常日、分门店/分类别稳健性和随机森林置换重要性作为辅助。问题四将历史销量、类别和外部变量纳入 Ridge 综合模型，并以严格 7 日递推验证作为主评价口径。

结果显示，问题一日滚动验证下门店主模型 $WAPE=37.49\%$ ，商品主模型 $WAPE=38.14\%$ ；问题二类别聚合后直接预测 $WAPE=34.74\%$ ；问题三主回归 $WAPE=84.53\%$ 。问题四严格 7 日递推验证下，问题一门店-商品简单指数平滑 $WAPE=76.35\%$ ，综合 Ridge $WAPE=76.68\%$ 。进一步的消融与低销量鲁棒性检验表明，低销量序列使用指数平滑、常规序列使用综合 Ridge 的混合策略 $WAPE=75.85\%$ ，较两个单一模型略有改善。有限优化中近 28 日均值兜底候选在同一验证集上 $WAPE$ 更低，但留一窗口检查未达到稳定替换阈值，因此本文仍将低销量指数平滑-常规 Ridge 混合策略作为候选最终预测方案。预测表 `outputs/final_7day_forecast_hybrid_low_volume.csv` 是在历史同期天气和活动日情景下得到的条件预测，原 `outputs/final_7day_forecast.csv` 保留为完整 Ridge 对照。

关键词：销量预测；移动平均；指数平滑；固定效应回归；严格递推验证； $WAPE$

1 问题重述

1.1 背景

休闲零食连锁门店需要根据历史销售数据安排订货、库存和促销活动。不同门店的客流基础不同，不同商品的销量规模和波动性也不同；同时，天气、节假日和门店活动可能与销售变化相关。因此，本题需要在理解历史销量结构的基础上，给出未来 7 天的销量预测，并分析商品关联和外部因素。

1.2 需要解决的问题

问题一要求分析不同门店和不同零食的销量情况，并分别预测未来 7 天总销量。该问题的输入是历史销售明细，输出包括门店层面、商品层面和门店-商品组合层面的预测结果。

问题二要求分析同一门店内不同零食销量之间的联系，整合同类零食，并预测未来 7 天总销量。该问题的关键是区分“商品销量同步波动”和“真实业务类别聚合”：相关性用于分析联系，附件类别字段用于正式类别预测。

问题三要求分析天气、节假日、活动日等因素与销量之间的关系。由于数据是观察数据而非随机实验，本文只讨论统计关联，不把外部变量写成严格因果影响。

问题四要求结合前三问模型和结论，预测各门店各种零食未来 7 天总销量，并与前序模型比较误差。该问题的关键是使用与未来 7 天预测一致的严格递推验证口径，避免把日滚动一步验证误写成未来 7 天一次性预测效果。

1.3 数据说明

附件一为历史零售明细，原始表含 67084 行、14 列，字段包括门店编号、门店名称、商品代码、商品名称、数量、销售额、收款时间和类别名称等。附件二为天气、节假日和促销活动数据，含 760 行、9 列，字段包括日期、天气、最高温、最低温、风力、节日、活动日和工休日等。字段说明见项目数据字典 DATA_DICTIONARY.md。

2 问题分析

本题的核心不是单一模型选择，而是按题目四问建立互相衔接的分析框架。问题一先建立可解释的销量基准模型，形成门店、商品和门店-商品三个粒度的基础预测；问题二在商品关联分析基础上进行类别聚合，解决同类零食整合预测；问题三分析外部变量与销量之间的条件统计关联，为综合模型提供特征解释；问题四在门店-商品粒度上融合历史销量、日历、类别和外部变量，并用严格 7 日递推验证检验其是否超过强基准模型。

预测评价不能随机切分，因为随机切分会让未来日期信息进入训练过程。本文使用时间顺序切分、日滚动验证和严格 7 日递推窗口验证。日滚动一步验证评价“每天更新真实销量后预测下一天”的能力；严格 7 日递推验证评价“窗口开始时一次性向未来预测 7 天”的能力，两者不能混用。

模型选择遵循三条原则：第一，所有结论来自附件数据或明确的模型计算；第二，优先使用本科数学建模竞赛中可解释、可复现、可写入论文的方法；第三，复杂模型只有在误差稳定下降且解释成本可控时才作为主方案。最终方法选择依据见 `outputs/method_search/final_method_selection.md`。

3 模型假设与符号说明

3.1 模型假设

1. 题目提供的销售明细和外部变量记录总体真实可靠。对于已发现的数据问题，本文采用记录、标记和统一口径处理，而不是直接删除原始数据。
2. 短期内历史销售规律具有一定延续性。未来 7 天预测依赖最近历史销量、同星期规律和类别/门店稳定差异。
3. 负销量不代表顾客正常购买需求。根据人工确认，负销量多对应破损、过期或冲销类负向调整，因此预测目标采用正向销量。
4. 附件类别字段可以作为同类零食的正式聚合依据。相关性聚类 and 销量规模分组只作为补充分析，不替代业务类别。
5. 天气、节假日和活动日与销量之间可以存在统计关联，但当前数据不足以识别严格因果效应。本文所称“影响因素分析”均指统计关联分析。
6. 未来 7 天无附件真实天气和真实活动安排。最终预测若使用天气和活动变量，应解释为“给定历史同期外部变量情景下”的条件预测。

3.2 符号说明

表 1: 主要符号说明

符号	含义
t	日期序号
s	门店编号
p	商品编号
g	商品类别
$y_{s,p,t}$	门店 s 、商品 p 在日期 t 的正向销量
$\hat{y}_{s,p,t}$	对 $y_{s,p,t}$ 的预测值
$Y_{t,g}$	日期 t 类别 g 的聚合销量
S_t	指数平滑中的平滑水平
α	指数平滑系数
r_{pq}	商品 p 与商品 q 的销量相关系数
X_t	日历、天气、活动日等外部变量
MAE	平均绝对误差
$RMSE$	均方根误差
$WAPE$	加权绝对百分比误差

4 数据预处理

4.1 数据检查

销售数据日期范围为 2020-04-01 至 2022-03-31，含 7 家门店、12 个商品代码、13 个商品名称和 8 个类别。商品代码 11001020 对应两个海苔商品名称，经人工确认以商品编号为准，名称差异视为录入笔误。天气数据日期范围为 2020-03-01 至 2022-03-30，不能覆盖销售最后一天 2022-03-31 和未来 2022-04-01 至 2022-04-07。

数据质量检查发现：销售明细中完全重复行 16605 条，经人工确认保留；负销量记录 54 条，作为损耗/冲销类负向调整；交易级数量范围为 -24 至 2712，IQR 规则标记 5536 条交易级极端记录。本文不删除原始记录，而是在处理表中保留正向销量、负向调整和异常标记。详细检查结果见 `outputs/stage1_data_inspection_report.md`。

4.2 日销量面板构造

本文将交易明细聚合为日期-门店-商品日销量面板，并补齐出现过的门店-商品组合在完整日期上的零销量记录。输出表 `data/processed/daily_store_product_sales.csv` 含 59130 行，日期范围为 2020-04-01 至 2022-03-31。合并外部变量后得到 `data/processed/modeling_base_table.csv`，同样为 59130 行。

日销量字段包括：

- `daily_sales`: 日净销量，包含负向调整；
- `positive_sales`: 正向销售数量，作为主预测目标；
- `negative_adjustment_qty`: 负向调整数量绝对值；
- `has_external_data`: 是否成功匹配附件二外部变量。

4.3 外部变量处理

附件二的“温度”和“温度.1”经人工确认为最高温和最低温，分别记为 `max_temperature` 和 `min_temperature`；“活动日”确认为门店促销活动日；“工休日”表示周末/节假日等休息日。缺少外部变量的销售日期为 2022-03-31，对应合并后 81 行 `has_external_data=0`。未来 7 天没有附件真实天气和活动日观测，本文在问题四中采用历史同期情景，并明确其条件预测性质。

5 问题一：门店与商品销量分析及未来 7 天预测

问题一以正向销量为目标，分别在门店、商品和门店-商品层级建模。设某一序列第 t 天销量为 y_t 。

5.1 模型建立

7 日移动平均模型为

$$\hat{y}_{t+1} = \frac{1}{7} \sum_{i=0}^6 y_{t-i}. \quad (1)$$

该模型用最近 7 天平均销量代表短期水平，适合作为门店层面的主模型。

同星期均值模型为

$$\hat{y}_t = \frac{1}{m} \sum_{j=1}^m y_{t-7j}, \quad m \leq 8. \quad (2)$$

该模型用历史相同星期几销量预测目标日，作为检验星期效应的基准模型。

简单指数平滑模型为

$$S_t = \alpha y_t + (1 - \alpha) S_{t-1}, \quad \hat{y}_{t+1} = S_t, \quad 0 < \alpha \leq 1. \quad (3)$$

该模型更重视近期销量，适合作为商品和门店-商品层面的主模型 [4]。

5.2 验证结果

阶段 2 日滚动验证结果见表 2。

表 2: 问题一日滚动验证下主模型误差

层级	主模型	MAE	RMSE	WAPE
门店	移动平均 (7 日)	9.685	15.445	37.49%
商品	简单指数平滑	5.748	10.968	38.14%
门店-商品	简单指数平滑	1.783	3.733	74.95%

为了使评价更贴近“未来 7 天总销量”，方法复核中补充 7 日窗口总量验证。结果显示，原问题一主方案在商品层面 WAPE=21.51%，门店层面 WAPE=22.12%，门店-商品层面 WAPE=43.06%。更复杂的滞后 Ridge 和随机森林未形成稳定优势，因此问题一保留原主模型。

5.3 描述性图表

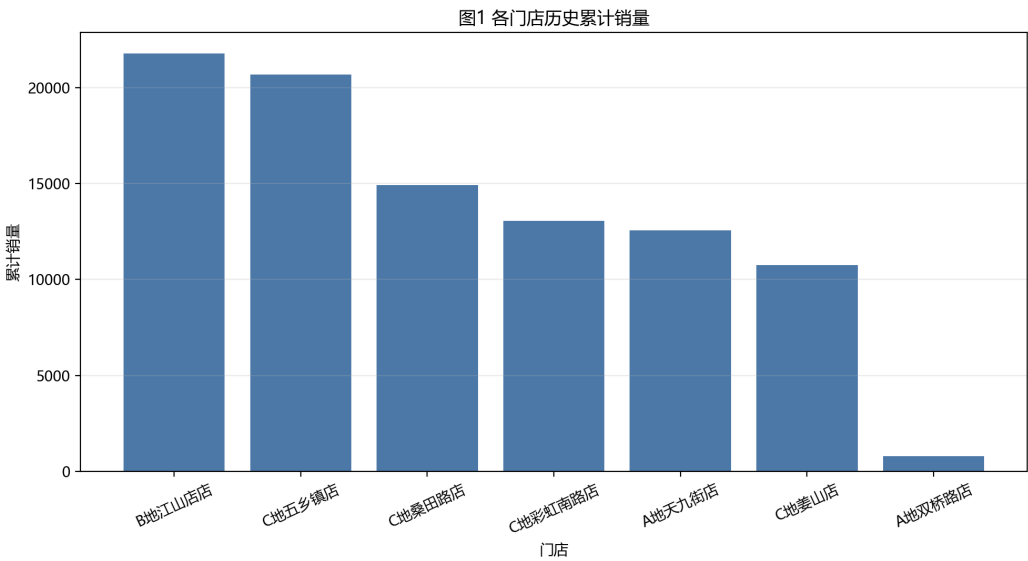


图 1: 各门店累计销量，用于说明门店销量规模差异

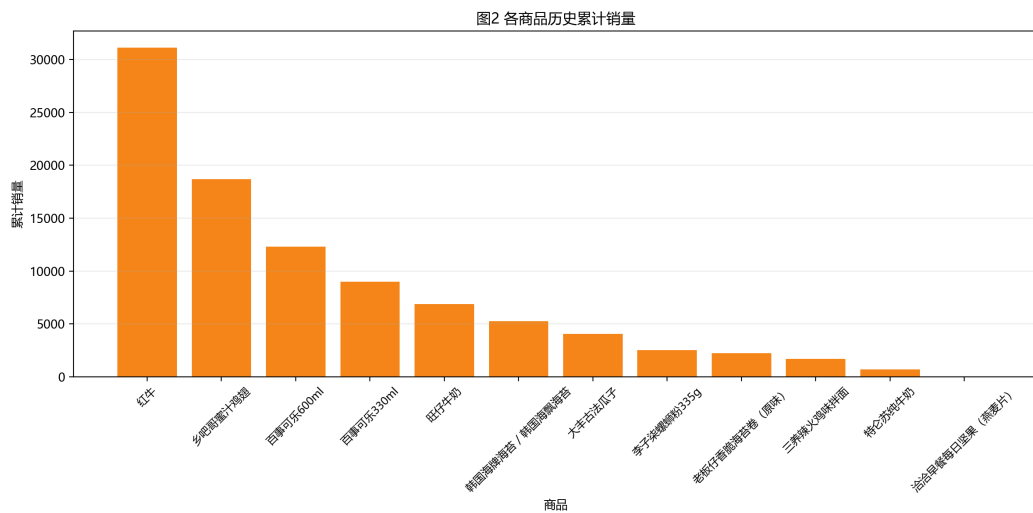


图 2: 各商品累计销量，用于说明商品销量规模差异

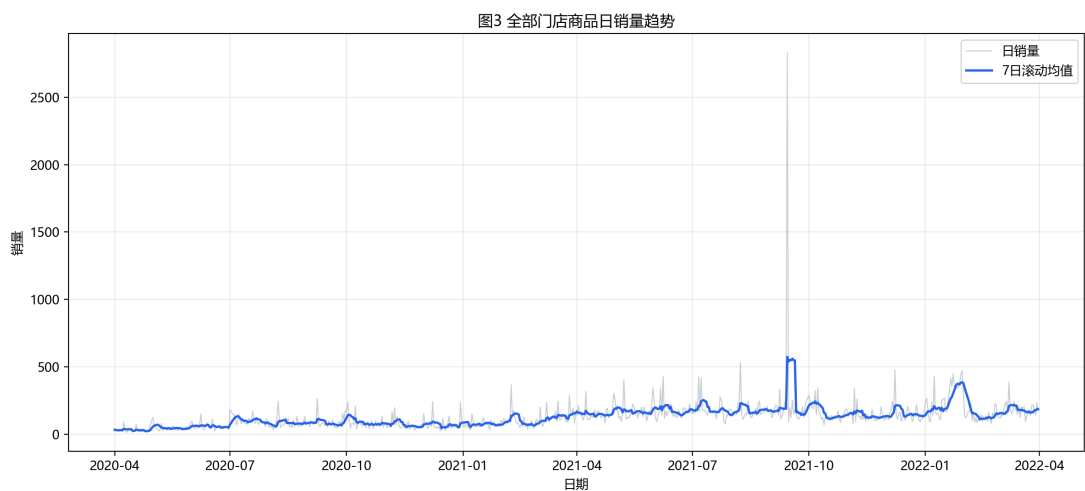


图 3: 总体日销量趋势，用于观察历史销量波动

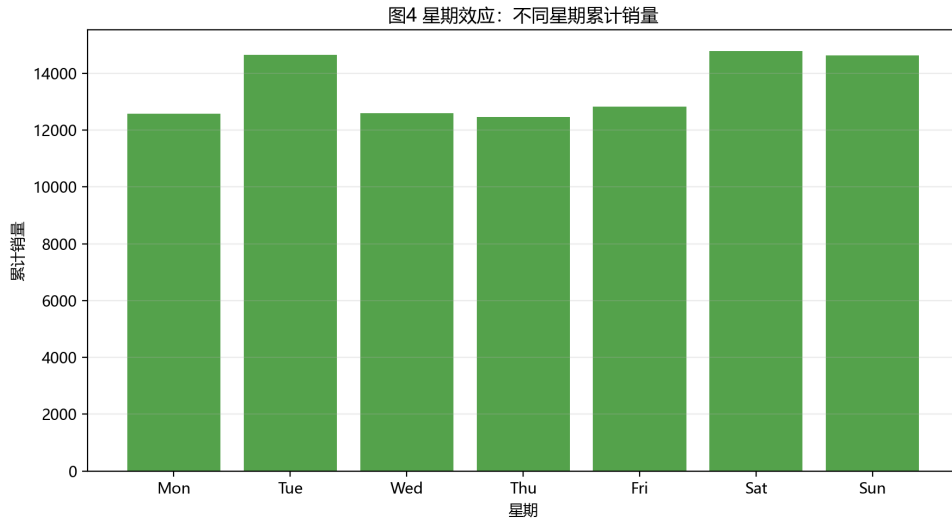


图 4: 星期效应，用于判断同星期均值模型是否有必要

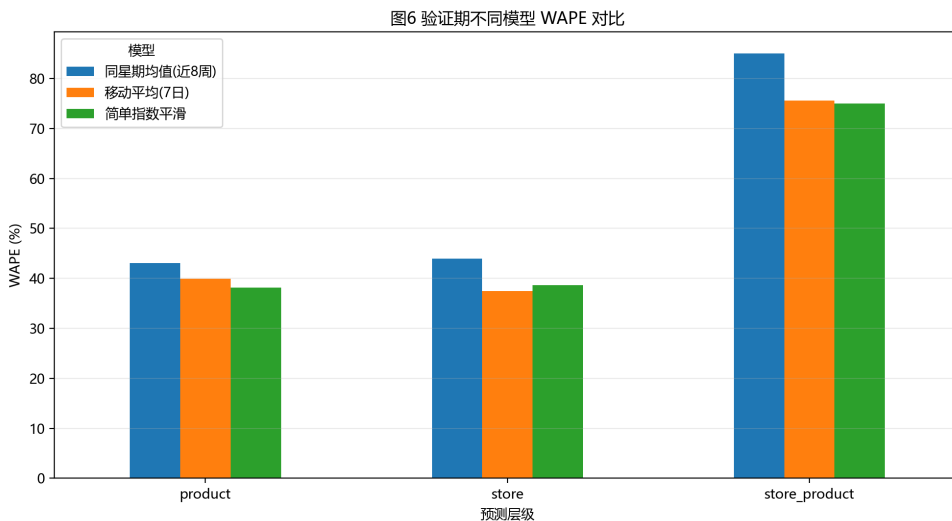


图 5: 问题一模型 WAPE 对比，用于选择各层级主模型

5.4 问题一结果

历史累计销量最高的门店为 B 地江山店，累计销量 21773；历史累计销量最高的商品为红牛，累计销量 31119。门店和商品销量规模差异明显，说明不能只用统一平均水平预测所有对象。

未来 7 天门店预测中，B 地江山店预测总销量最高，为 309.0；C 地五乡镇店为 280.0；A 地天九街店为 245.0。商品预测中，乡吧哥蜜汁鸡翅预测 389.613，红牛预测 286.719，百事可乐 600ml 预测 136.219。问题一门店层面和商品层面分别建模，预测总量没有强制一致：门店层面合计 1296.000，商品层面合计 1226.006。

6 问题二：商品关联分析与类别聚合预测

问题二关注同一门店中不同零食销量之间的联系，并要求整合同类零食预测未来 7 天总销量。设 $y_{t,p}$ 表示第 t 天商品 p 的正向销量， $c(p)$ 表示商品所属类别。

6.1 商品销量矩阵与相关性

对每个门店构造日期 $\times$ 商品销量矩阵：

$$Y_s = (y_{t,p}), \quad t = 1, \dots, n, \quad p = 1, \dots, m. \quad (4)$$

商品 p 与商品 q 的 Pearson 相关系数为

$$r_{pq} = \frac{\sum_t (y_{t,p} - \bar{y}_p)(y_{t,q} - \bar{y}_q)}{\sqrt{\sum_t (y_{t,p} - \bar{y}_p)^2} \sqrt{\sum_t (y_{t,q} - \bar{y}_q)^2}}. \quad (5)$$

为避免单一 Pearson 相关造成过度解释，本文补充 Spearman 相关、去星期效应 Pearson 和门店内相关稳定性检查。商品关联结论只解释为同步波动，不写成果或替代关系。

图7 全部门店汇总后的商品销量相关性热力图

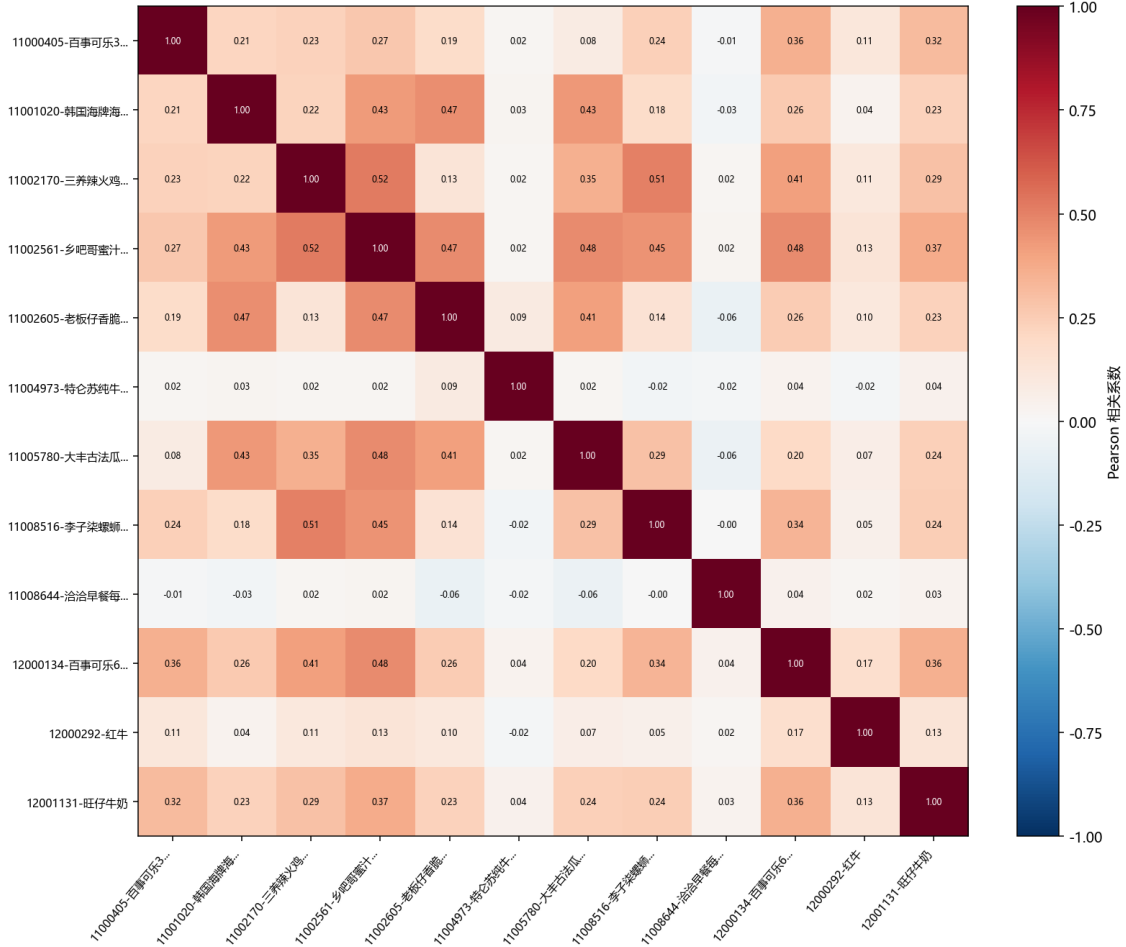


图 6: 商品相关性热力图，用于展示商品日销量同步波动结构

6.2 同类零食整合

类别聚合按附件 category 字段进行：

$$Y_{t,g} = \sum_{p:c(p)=g} y_{t,p}. \quad (6)$$

该字段具有业务含义，因此作为“同类零食”的正式聚合依据。相关性聚类 and 销量规模分组可作为补充探索，但不替代附件业务类别。

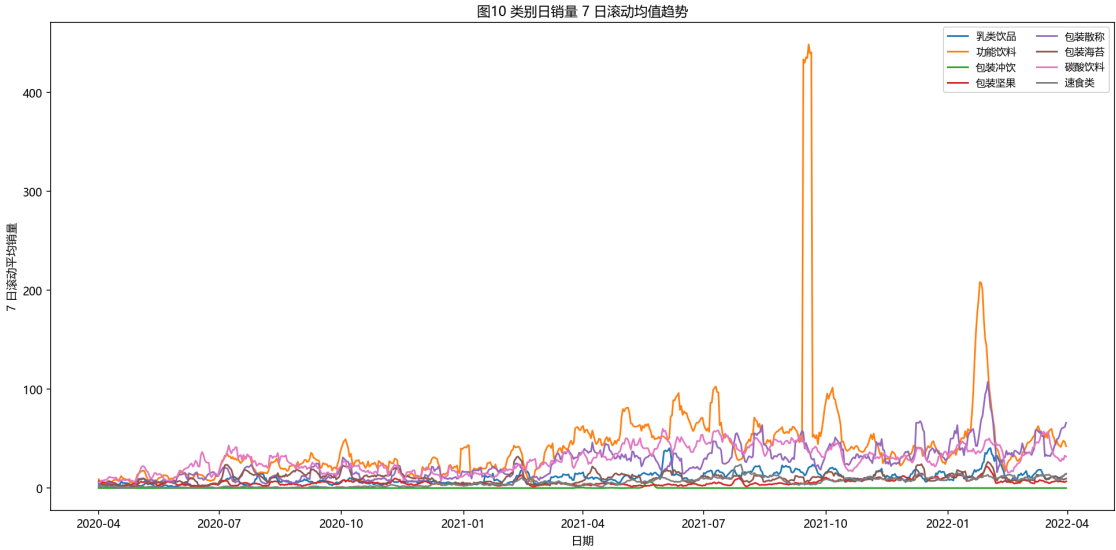


图 7: 类别日销量趋势，用于观察类别层面的时间波动

6.3 类别预测与误差比较

本文比较“单品预测后按类别加总”和“类别聚合后直接预测”。验证结果见表 3。

表 3: 问题二类别预测策略误差比较

方法	模型	MAE	RMSE	WAPE
类别聚合后直接预测	简单指数平滑	7.853	13.933	34.74%
单品预测后按类别加总	简单指数平滑	7.873	13.949	34.83%
类别聚合后直接预测	移动平均 (7 日)	8.246	14.174	36.48%
类别聚合后直接预测	同星期均值 (近 8 周)	8.851	15.057	39.15%

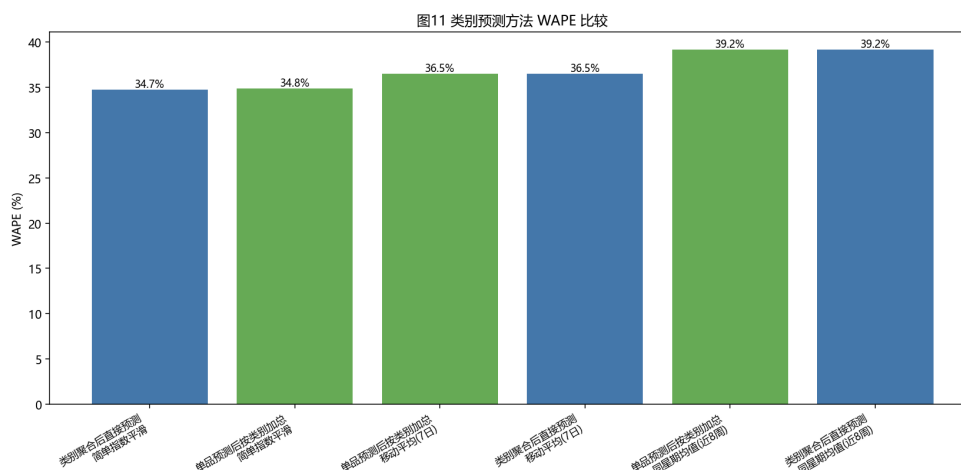


图 8: 类别预测方法 WAPE 对比，用于比较聚合后直接预测和单品预测后加总

因此，问题二主方案采用“附件类别聚合后直接预测 + 简单指数平滑”。相关性聚类 WAPE=31.02%，销量规模分组 WAPE=24.24%，但前者业务类别含义弱，后者预测对象变成高/中/低销量组，均不替代附件类别主方案。

6.4 问题二结果

商品关联分析表明，全部门店汇总层面存在 2 对强正相关商品对和 25 对弱相关商品对。强正相关只能说明历史日销量同步波动，可能共同受客流、消费场景或促销节奏影响，不能证明商品之间存在直接带动关系。负相关候选的相关系数绝对值较小，当前数据未发现强替代关系证据。

类别预测结果显示，未来 7 天预测最高的类别为包装散称，预测 389.613；其次为功能饮料，预测 286.719；碳酸饮料预测 247.326。类别聚合可以降低单品零销量和偶然大单带来的波动，但会损失同一类别内部商品差异。

7 问题三：外部因素统计关联分析

问题三分析天气、节假日、活动日等外部因素与零食销量之间的统计关联。本文将外部因素构造为分类变量和连续变量：天气类型合并为更稳定的天气组，最高温、最低温、平均温度、昼夜温差和风力为连续变量，节假日、周末、活动日为 0-1 变量，星期、月份、门店、商品和历史销量作为控制变量。原始附件中没有湿度字段，因此不构造湿度变量。

7.1 描述性分析

对某一外部因素 G ，先计算不同分组下的平均销量：

$$\bar{y}_g = \frac{1}{n_g} \sum_{i:G_i=g} y_i. \quad (7)$$

该方法直观，但不能控制门店、商品和星期等混杂因素，只作为现象展示。

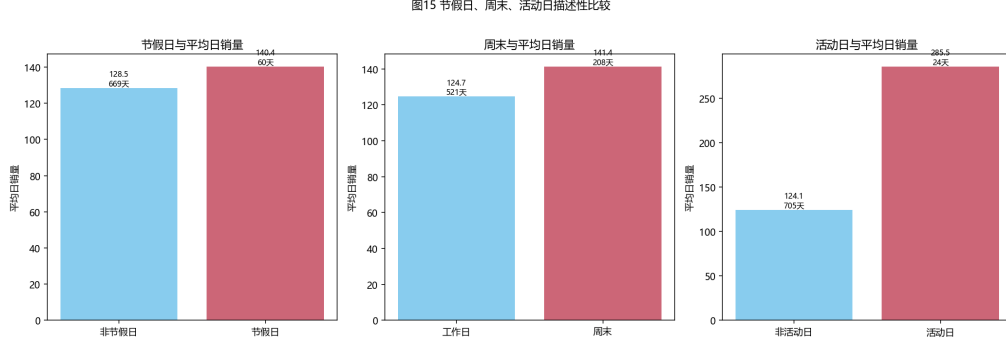


图 9: 二元外部因素均值差异，用于展示节假日、周末和活动日下的描述性销量差异

7.2 固定效应回归模型

问题三主模型采用“合并天气 + 对数销量 + 历史控制固定效应回归”。设 $z_{s,p,t} = \log(1 + y_{s,p,t})$ ，模型形式为

$$z_{s,p,t} = \beta_0 + \beta^\top External_t + \rho_1 lag7_{s,p,t} + \rho_2 rolling28_{s,p,t} + \gamma_s + \delta_p + \eta_w + \mu_m + \varepsilon_{s,p,t}. \quad (8)$$

其中， $External_t$ 包括合并天气类别、平均温度、昼夜温差、风力、节假日和活动日； γ_s 、 δ_p 、 η_w 、 μ_m 分别表示门店、商品、星期和月份固定效应； lag_7 和 $rolling_28_prev$ 只使用过去销量。线性回归和固定效应思想参考 [5]。

选择该模型的原因是：详细天气类别中存在雨夹雪、暴雨等少数样本天气，直接排序容易不稳；原始销量分布偏斜，对极端销量敏感；加入历史控制项后能减少基础需求差异对外部变量系数的干扰。

方法探索验证指标见表 4。

表 4: 问题三外部因素分析方法验证指标

方法	MAE	RMSE	WAPE	论文定位
原方案详细天气 FE 回归	1.969	4.144	88.09%	补充
合并天气 + 对数销量 + 历史控制 FE 回归	1.890	4.448	84.53%	主分析
温度分箱 + 对数销量 + 历史控制 FE 回归	1.889	4.445	84.51%	非线性检查
去异常日 + 对数销量 + 历史控制 FE 回归	1.825	4.254	84.94%	稳健性检查
随机森林 grouped weather + history	1.892	4.178	84.66%	预测贡献补充

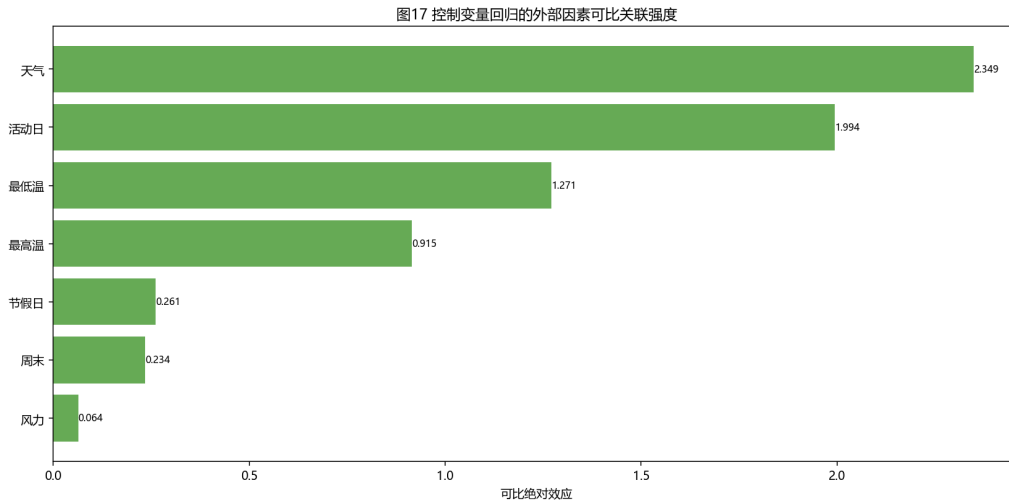


图 10: 回归关联强度排序，用于展示外部变量在控制部分混杂因素后的可比统计关联

7.3 机器学习辅助分析

随机森林置换重要性用于补充判断非线性预测贡献 [1]。它能说明打乱某一特征后预测误差增加多少，但不能说明销量变化方向，也不能证明因果关系。因此，问题三最终结论以回归的统计关联方向为主，以随机森林重要性作为辅助排序依据。

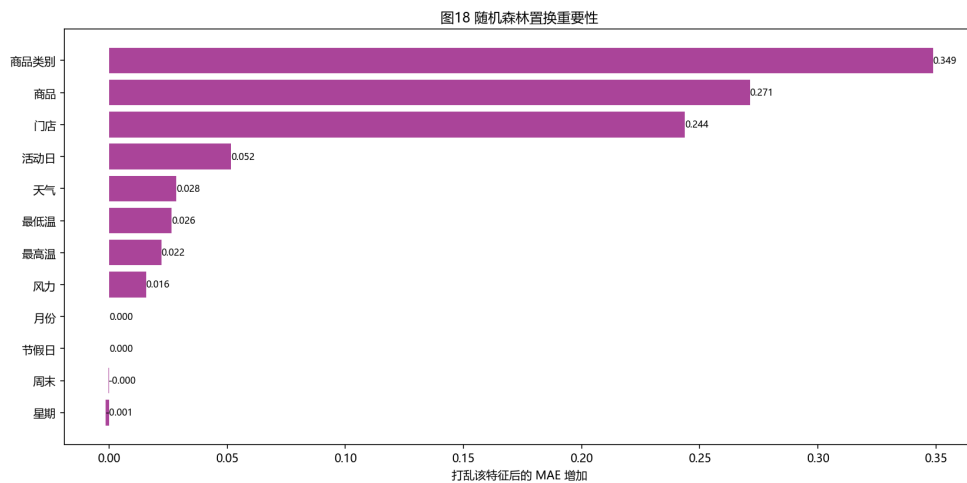


图 11: 随机森林置换重要性，用于展示非线性模型中的预测贡献

7.4 问题三结果

描述性统计显示，不同天气、节假日、周末和活动日下的平均销量存在差异，但这些差异可能与门店、商品、星期和月份同时相关。固定效应回归和稳健性检查表明，活动日变量与销量上升之间的统计关联相对更稳定；天气变量存在一定关联，但细分天气类别受样本量限制；节假日和周末与星期效应、月份季节性重叠，结论应作为辅助。本文不分析湿度，因为附件中没有湿度字段。

8 问题四：综合预测模型与严格递推验证

问题四需要在门店-商品粒度上进行综合预测。设 $y_{s,p,t}$ 表示门店 s 、商品 p 在日期 t 的正向销量，综合模型的特征包括历史销量滞后项、滚动均值、日历变量、门店、商品、类别和外部变量。

8.1 综合模型构造

综合 Ridge 的特征包括滞后销量、滚动均值、星期、月份、是否周末、门店、商品、类别、天气、节假日和活动日。设 $x_{s,p,t}$ 为上述变量经数值化和独热编码后的特征向量，Ridge 回归估计式为

$$\hat{y}_{s,p,t} = x_{s,p,t}^\top \hat{\beta}, \quad (9)$$

其中参数由如下正则化最小二乘问题确定：

$$\hat{\beta} = \arg \min_{\beta} \left\{ \sum_{(s,p,t) \in \mathcal{T}} (y_{s,p,t} - x_{s,p,t}^\top \beta)^2 + \lambda \sum_{j=1}^d \beta_j^2 \right\}. \quad (10)$$

其中， $\mathcal{T}$ 为训练样本集合， λ 为正则化强度， d 为特征维数。*Lag* 包括 `lag_1`、`lag_7`、`lag_14`，*Roll* 包括 `rolling_mean_7` 和 `rolling_mean_14`，*Cal* 包括星期、月份和周末标记，*External* 包括天气、温度、风力、节假日和活动日。完整字段含义见 `outputs/q4_feature_dictionary.csv`。Ridge 回归通过 L_2 正则化缓解多重共线性，适合在保留线性可解释性的同时融合多类特征 [3]。随机森林和其他树模型只作为补充比较，不作为论文主模型。

8.2 严格 7 日递推验证

信息泄露专项检查 `outputs/q4_information_leakage_review.md` 指出：原日滚动验证没有当天销量直接进入当天特征，但验证窗口后续日期的 `lag_1` 和滚动均值会使用窗口内前几天真实销量。因此，问题四以 `outputs/q4_recursive_7day_model_comparison.md` 中的严格 7 日递推验证作为主口径。

严格递推验证窗口见表 5。每个窗口只使用窗口起点以前的真实历史销量训练模型；窗口内第 2 至第 7 天若需要滞后或滚动特征，则使用前面已经预测出的销量，而不使用窗口内真实销量。

表 5: 问题四严格 7 日递推验证窗口

窗口起点	窗口终点	长度
2022-03-01	2022-03-07	7
2022-03-08	2022-03-14	7
2022-03-15	2022-03-21	7
2022-03-22	2022-03-28	7

严格递推下，门店-商品层级误差见表 6。

表 6: 问题四严格 7 日递推主粒度误差比较

模型	MAE	RMSE	WAPE
问题一门店-商品简单指数平滑	1.816	3.772	76.35%
综合 Ridge 回归	1.824	3.702	76.68%
14 日滚动均值基准模型	1.847	3.819	77.66%
7 日移动平均基准模型	1.865	3.797	78.43%
综合随机森林	1.988	4.842	83.60%

在严格 7 日递推验证口径下，综合 Ridge 模型相较部分基准模型仍具有竞争力，但未超过问题一简单指数平滑模型。其主要价值在于统一融合门店、商品、类别、历史销量和外部因素，为多因素解释和情景预测提供一致框架。本文不作全局最优或显著提升的夸大表述。

8.3 消融检验与低销量混合策略

为进一步说明综合模型中不同特征的贡献，本文在相同严格 7 日递推窗口下进行消融检验，结果见表 7。消融结果表明，去除天气变量后 WAPE 从 76.68% 上升至 77.73%，去除活动日后上升至 77.22%，不使用销量历史时上升至 84.49%。这说明销量滞后和滚动统计仍是最核心的信息，天气和活动日具有一定预测补充作用，但不能解释为因果影响。

表 7: 问题四严格递推消融检验

模型	MAE	RMSE	WAPE
问题一门店-商品简单指数平滑	1.816	3.772	76.35%
完整综合 Ridge	1.824	3.702	76.68%
去节假日活动 Ridge	1.830	3.748	76.96%
去活动日 Ridge	1.837	3.748	77.22%
去天气 Ridge	1.849	3.698	77.73%
历史日历 Ridge	1.859	3.739	78.19%
无销量历史 Ridge	2.009	4.252	84.49%

同时，低销量诊断显示大量门店-商品序列存在零销量和间歇需求。本文不直接引入复杂 Croston/TSB 模型，而采用一条可解释的规则化混合策略：在预测窗口开始日前，若某门店-商品组合被判为低销量序列，则使用问题一简单指数平滑；其余常规序列使用完整综合 Ridge。低销量判定只使用窗口开始日前历史数据：若正销量天数占比不高于 0.15 或历史日均销量低于 0.20，则记为 `very_sparse`；若零销量比例不低于 0.75 或历史日均销量低于 0.50，则记为 `low_volume_intermittent`；无正销量历史序列单独记为 `no_positive_history`。上述三类进入低销量兜底，其余序列仍由 Ridge 预测。该策略的严格递推验证结果见表 8。

表 8: 低销量鲁棒性策略比较

模型	MAE	RMSE	WAPE
低销量指数平滑-常规 Ridge 混合	1.804	3.718	75.85%
问题一简单指数平滑	1.816	3.772	76.35%
低销量收缩-常规 Ridge 混合	1.823	3.709	76.66%
完整综合 Ridge	1.824	3.702	76.68%
低销量收缩指数平滑	1.835	3.764	77.15%

该混合策略相对问题一简单指数平滑降低 0.49 个百分点，相对完整 Ridge 降低 0.83 个百分点。由于改进幅度不大且验证窗口有限，本文将其表述为低销量场景下的稳健改进，而不夸大为显著突破。完整结果见 `outputs/q4_ablation_report.md` 和 `outputs/q4_low_volume_strategy_report.md`。

本轮进一步尝试了低销量阈值调整、近 28 日均值、商品均值、类别均值和层级收缩等兜底方法，并检查门店、商品和类别层级比例校准。全样本严格递推下，“低销量

用近 28 日均值”的探索性候选 WAPE 可降至 74.95%，但留一窗口选模后的平均改善仅 0.19 个百分点，未达到 0.3 个百分点的稳定替换阈值。因此，本文不把该探索候选替换为主方案，只将其作为附加稳健性检查。详细结果见 `outputs/q4_candidate_model_review.md`。

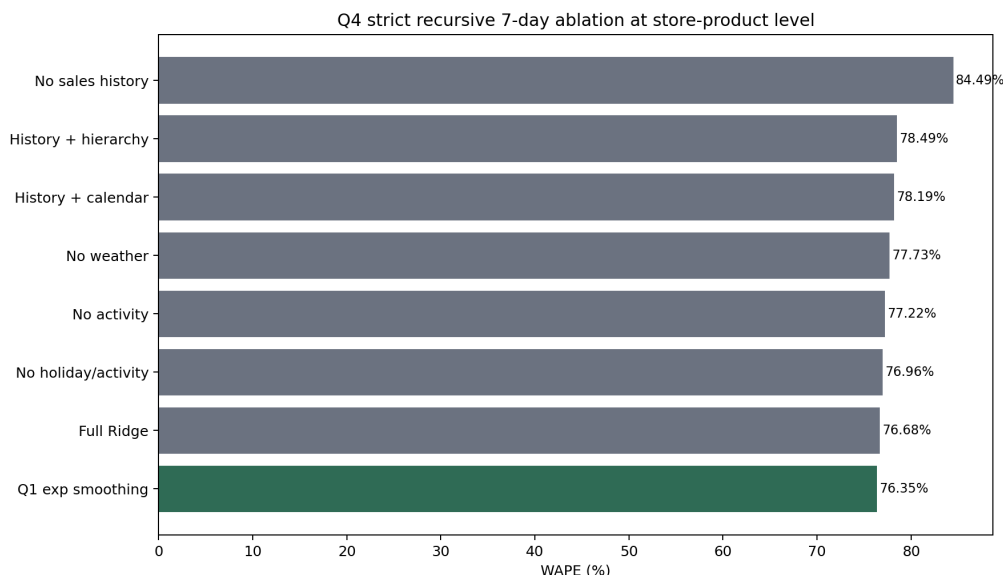


图 12: 严格递推消融检验 WAPE 对比，用于说明历史销量和外部变量的边际贡献

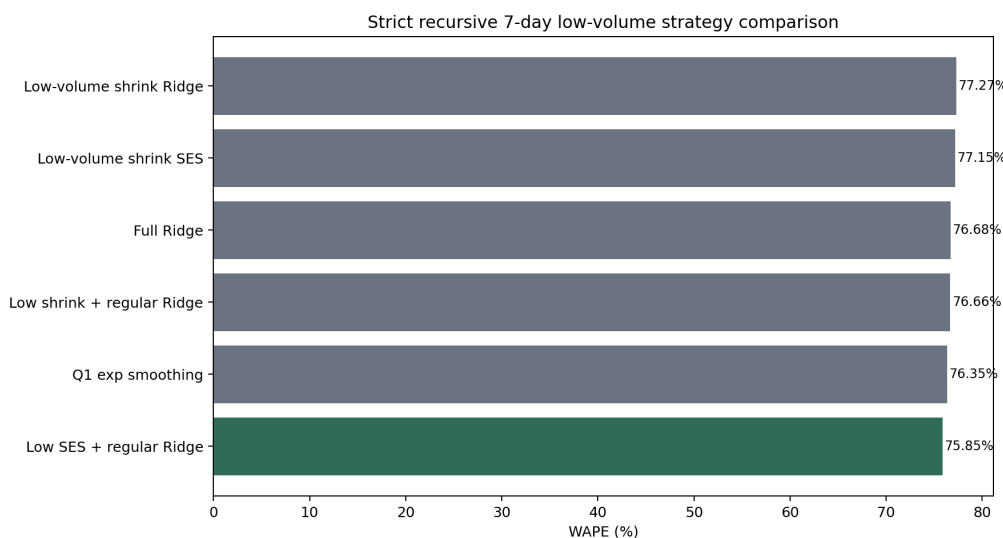


图 13: 低销量鲁棒性策略 WAPE 对比，用于说明混合策略的验证效果

8.4 未来 7 天预测与整数化

未来 7 天天气、温度、风力和活动日没有附件真实观测，最终预测使用历史同期参考情景；清明节日历按 2022-04-03 至 2022-04-05 为节假日、2022-04-02 为调休上班日处理。因此，最终预测为给定外部变量情景下的条件预测。

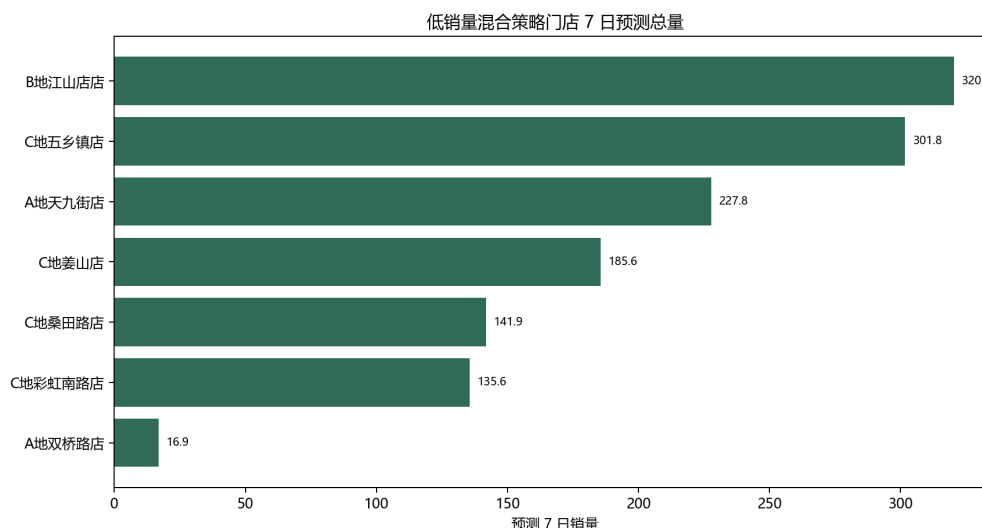


图 14: 低销量混合策略下未来 7 天门店预测总量，用于展示推荐预测表在门店层面的分布

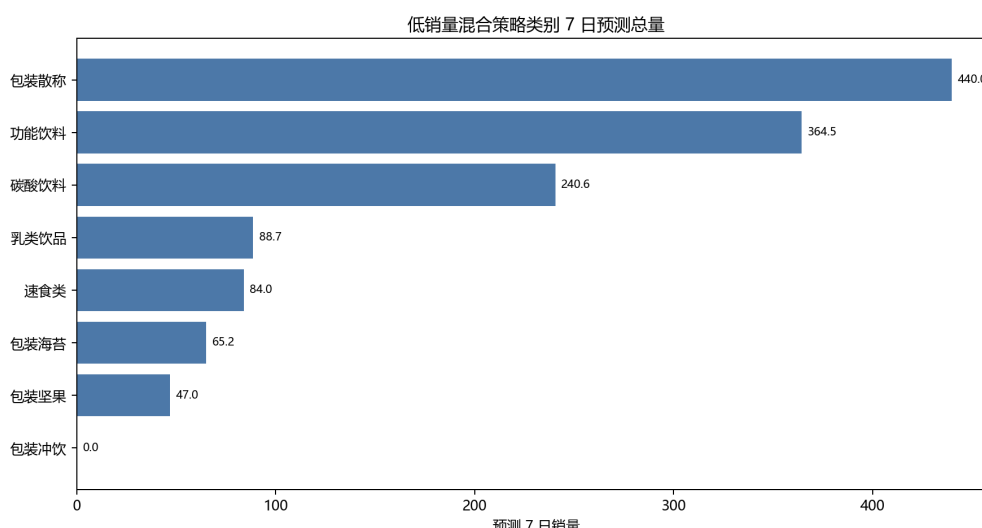


图 15: 低销量混合策略下未来 7 天类别预测总量，用于展示推荐预测表在类别层面的分布

未来 7 天低销量混合策略条件预测中，各门店预测总销量最高的是 B 地江山店 320.342，其次为 C 地五乡镇店 301.763 和 A 地天九街店 227.846；各类别预测最高的是包装散称 439.965，其次为功能饮料 364.481 和碳酸饮料 240.589。图 14 和图 15 均由同一混合策略预测表生成，与下述预测总量一致。完整门店-商品-日期小数预测表见 `outputs/final_7day_forecast_hybrid_low_volume.csv`；原完整 Ridge 预测表 `outputs/final_7day_forecast.csv` 保留为对照。

由于实际销量以件数计，提交展示时对模型预测值进行非负约束和四舍五入整数化处理：预测值小于 0 的先置为 0，极小预测值低于 10^{-6} 时置为 0，其余按四舍五

入转为整数；原始小数预测结果仍作为模型连续输出保留。混合策略整数化预测表为 `outputs/final_7day_forecast_hybrid_low_volume_integer.csv`。混合策略小数预测表 7 天预测总量为 1329.894，整数化后 7 天预测总量为 1328；原完整 Ridge 小数预测总量为 1311.294。活动日变量与天气变量一样属于历史同期情景假设，不代表已知的真实未来促销安排。

8.5 误差诊断与情景敏感性

天气敏感性检验 `outputs/q4_weather_sensitivity_report.md` 显示，完整外部变量 Ridge 的严格递推 WAPE 为 76.68%，去除天气变量 Ridge 为 77.73%，历史同期天气情景 Ridge 为 76.57%。因此，天气变量可作为补充情景特征，但不能写成稳定因果因素。

误差诊断 `outputs/q4_error_diagnosis_report.md` 显示，综合 Ridge 在门店 A 地双桥路店、低销量商品和样本不足的活动日场景误差较大；间歇性需求审查 `outputs/intermittent_demand_review.md` 显示，76 条门店-商品序列中有 54 条零销量比例不低于 50%，解释了门店-商品层级 WAPE 较高的原因。间歇性需求建模思想可参考 Croston 方法 [2]，但本文不将其作为主模型。

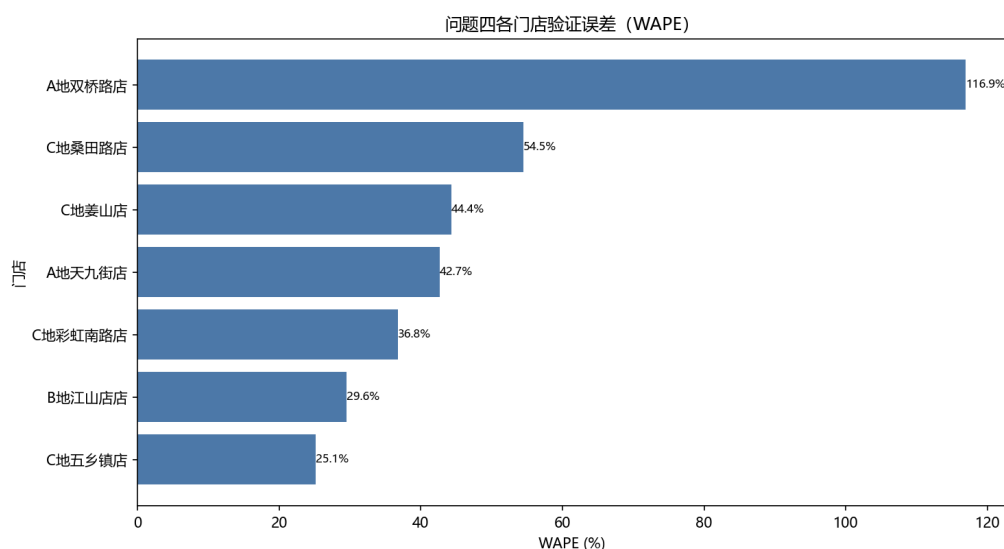


图 16: 门店误差诊断，用于定位严格递推验证下误差较高的门店

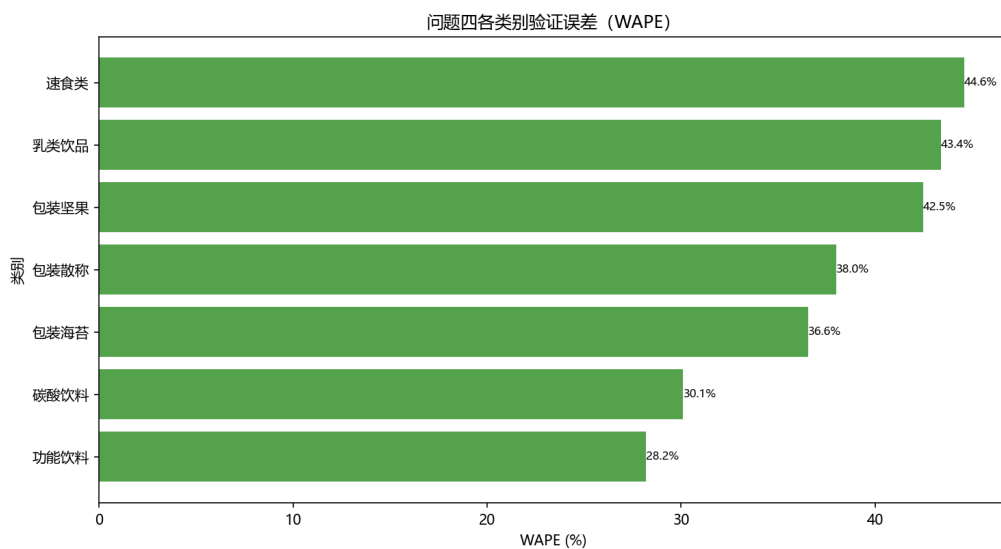


图 17: 类别误差诊断, 用于定位严格递推验证下误差较高的类别

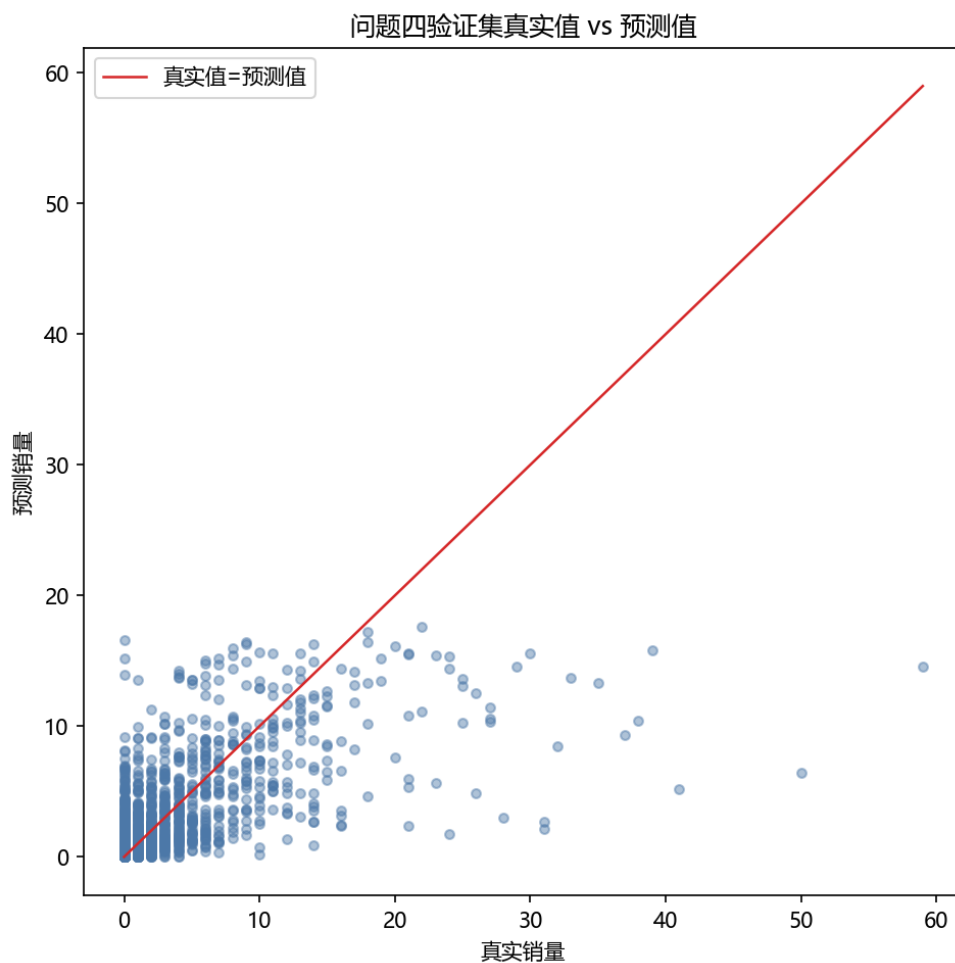


图 18: 严格递推验证真实值与预测值散点, 用于检查预测值与真实值的对应关系

9 模型评价

9.1 误差指标

本文主要采用 MAE、RMSE 和 WAPE 评价预测误差：

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (11)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (12)$$

$$WAPE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n |y_i|}. \quad (13)$$

MAE 表示平均绝对偏差，便于解释平均误差规模；RMSE 对大误差更敏感，适合检查极端预测偏差；WAPE 用总绝对误差除以总实际销量，适合本题中不同门店和商品销量规模差异较大的情况。对于真实销量为 0 的类别或商品，WAPE 无法定义，需要结合 MAE 和预测合计单独说明。

9.2 分问题评价

问题一中，日滚动验证结果显示，门店层面移动平均 (7 日) WAPE=37.49%，商品层面简单指数平滑 WAPE=38.14%，门店-商品层面简单指数平滑 WAPE=74.95%。补充的 7 日窗口总量验证表明，原问题一主方案在商品层面 WAPE=21.51%，门店层面 WAPE=22.12%，门店-商品层面 WAPE=43.06%。因此，问题一保留简单可解释模型，不引入复杂模型替换主方案。

问题二中，类别聚合后直接预测 + 简单指数平滑的 WAPE=34.74%，略优于单品预测后按类别加总的 WAPE=34.83%。销量规模分组 WAPE=24.24%，相关性聚类 WAPE=31.02%，但二者不直接回答“同类零食类别预测”要求，因此只作为补充，不作为主方案。

问题三中，合并天气、对数销量和历史控制固定效应回归的验证 WAPE=84.53%，优于原详细天气固定效应回归的 WAPE=88.09%。随机森林 grouped weather + history 的 WAPE=84.66%，可作为预测贡献补充，但不提供清晰方向，也不能解释因果。问题三主结论应写成“条件统计关联”。

问题四中，严格 7 日递推验证是主评价口径。该口径下，问题一门店-商品简单指数平滑 WAPE=76.35%，综合 Ridge WAPE=76.68%，14 日滚动均值基准模型 WAPE=77.66%，7 日移动平均基准模型 WAPE=78.43%。综合 Ridge 优于问题四内部基准模型，但未超过问题一强基准模型。因此，本文先将其定位为融合前三问信息的解释型综合模型。进一步的低销量鲁棒性检验显示，低销量序列使用指数平滑、常规序列使用综合 Ridge 的混合策略 WAPE=75.85%，相对问题一强基准降低 0.49 个百分点。

点；该结果说明细粒度稀疏需求需要分层处理，但由于改进幅度有限，仍应克制表述为稳健性改进。本轮有限优化还比较了不同低销量阈值、近 28 日均值兜底和层级比例校准；虽然近 28 日均值候选在同一验证集上 WAPE=74.95%，但留一窗口选模平均改善仅 0.19 个百分点，未达到稳定替换阈值，因此不替换当前推荐预测表。

9.3 不确定性

不确定性主要来自四类来源。第一，未来天气和活动日采用历史同期情景，若实际天气或促销安排偏离该情景，最终预测会变化。第二，低销量门店-商品序列存在大量零销量，当前混合策略下低销量序列整体 WAPE 约为 122.53%，少量绝对误差即可被低销量分母放大；混合策略虽有所改善，但并未消除稀疏需求带来的不确定性。第三，部分稀有天气样本天数较少，例如雨夹雪、暴雨等，不能把其系数作为稳定主结论。第四，严格递推验证只有 4 个完整 7 日窗口，模型选择结论仍需谨慎。

为辅助解释最终点预测，本文基于当前混合策略在 4 个严格 7 日递推验证窗口上的 7 日聚合残差构造经验区间。对每个门店、商品或类别，先计算“真实 7 日销量减预测 7 日销量”的残差，再取历史残差的 10% 和 90% 分位数加到未来点预测上，形成经验 80% 残差区间。该区间不是严格置信区间，只用于展示历史递推误差下的不确定性范围。门店级结果见图 19，完整表见 tables/final_forecast_interval_by_store.csv、tables/final_forecast_interval_by_product.csv 和 tables/final_forecast_interval_by_category.csv。

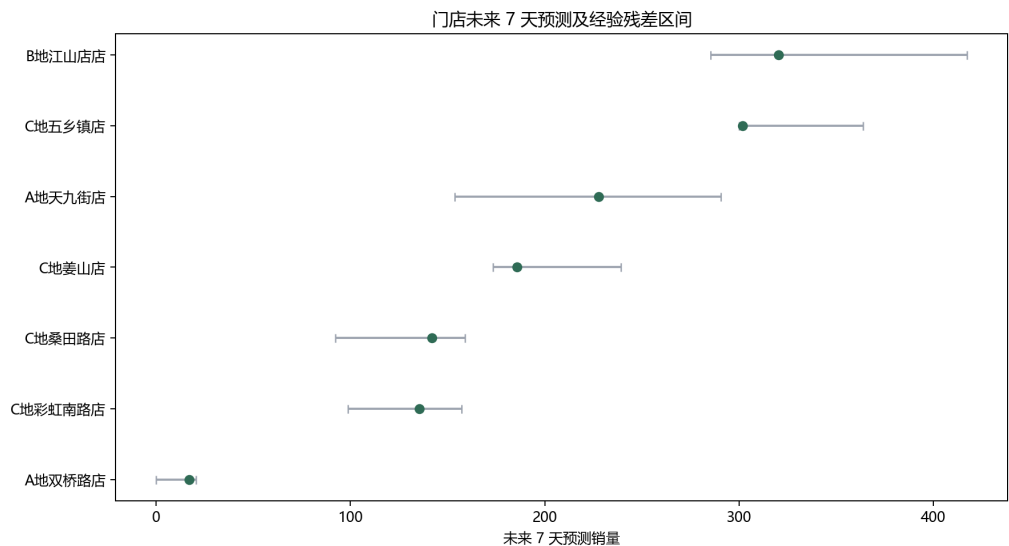


图 19: 基于严格递推残差的门店未来 7 天经验预测区间，用于提示点预测的不确定性

10 优缺点分析与结论

10.1 模型优点

本文模型具有以下优点：

1. 数据口径清晰，区分正向销量、净销量和负向调整。
2. 模型与题目四问一一对应，形成门店、商品、类别和门店-商品的分层预测框架。
3. 主模型优先采用移动平均、同星期均值、指数平滑和固定效应回归，公式简单、可解释、易复现。
4. 对综合模型进行了信息泄露审查，并以严格 7 日递推验证作为主口径。
5. 对外部变量和商品关联均设置了证据边界，没有把相关性写成果因果性。

10.2 模型局限

模型局限包括：第一，门店-商品粒度存在大量零销量和间歇性需求，导致 WAPE 较高；第二，未来真实天气和活动安排不可得，含外部变量预测依赖历史同期情景；第三，附件没有顾客购物篮、价格实验、库存和陈列信息，因此无法识别商品互补、替代或促销因果效应；第四，验证期只有有限的完整 7 日窗口，显著性检验和稳定性判断应谨慎。

10.3 结论

本文完成了休闲零食连锁店商品销量预测的四问建模。问题一保留移动平均、同星期均值和简单指数平滑，建立了门店、商品和门店-商品的基础预测；问题二以附件类别字段整合同类零食，商品相关性分析作为同步波动证据，类别预测主方案为类别聚合后直接预测；问题三采用合并天气、对数销量和历史控制固定效应回归，得到外部变量与销量之间的条件统计关联；问题四采用综合 Ridge 模型融合前三问信息，并用严格 7 日递推验证检验未来 7 天预测能力。

最终结论是：简单可解释模型在本题中具有很强竞争力。问题四综合 Ridge 虽然在日滚动一步预测和部分聚合层级上有数值优势，但在严格 7 日递推主粒度下未超过问题一门店-商品简单指数平滑强基准模型。因此，本文不夸大综合模型的预测提升，而将其作为融合历史销量、类别和外部变量的解释框架。进一步的低销量鲁棒性检验表明，对低销量组合采用指数平滑、对常规组合采用综合 Ridge 的混合策略在严格递推主粒度上 WAPE=75.85%，略优于两个单一模型。有限优化中近 28 日均值兜底候选在同一验证集上表现更低，但留一窗口检查未达到稳定替换阈值，因此本文仍推荐使用 `outputs/final_7day_forecast_hybrid_low_volume.csv` 作为候选最终预测表，同时保留 `outputs/final_7day_forecast.csv` 作为完整 Ridge 对照；两者均是在历史同期天气和活动日情景下的条件预测。混合策略整数化提交副本为 `outputs/final_7day_forecast_hybrid_low_volume_integer.csv`。

参考文献

- [1] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [2] J. D. Croston. Forecasting and stock control for intermittent demands. *Operational Research Quarterly*, 23(3):289–303, 1972.
- [3] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [4] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 3 edition, 2021.
- [5] Douglas C. Montgomery, Elizabeth A. Peck, and G. Geoffrey Vining. *Introduction to Linear Regression Analysis*. Wiley, 5 edition, 2012.

A 附录

A.1 主要代码文件

- 数据读取与清洗: `src/data_loader.py`、`src/preprocessing.py`
- 特征工程: `src/features.py`
- 问题二分析: `src/stage3_q2_analysis.py`
- 问题三分析: `src/stage4_q3_analysis.py`
- 问题四综合模型: `src/stage5_q4_analysis.py`
- 严格递推验证: `src/stage5_q4_recursive_validation.py`
- 问题四误差诊断: `src/stage5_q4_error_diagnosis.py`
- 天气敏感性分析: `src/stage5_q4_weather_sensitivity.py`
- 严格递推消融检验: `src/stage5_q4_ablation.py`
- 低销量鲁棒性策略: `src/stage5_q4_low_volume_strategy.py`
- 低销量混合最终预测: `src/stage5_q4_hybrid_final_forecast.py`
- 候选优化、不确定性和区间分析: `src/stage5_q4_award_iteration.py`
- 间歇性需求审查: `src/intermittent_demand_review.py`

A.2 主要结果文件

- 数据检查: `outputs/stage1_data_inspection_report.md`
- 问题一报告: `outputs/stage2_q1_modeling_report.md`
- 问题二报告: `outputs/stage3_q2_relationship_report.md`
- 问题三报告: `outputs/stage4_q3_factor_analysis_report.md`

- 问题四报告: outputs/stage5_q4_final_model_report.md
- 信息泄露审查: outputs/q4_information_leakage_review.md
- 严格递推验证: outputs/q4_recursive_7day_model_comparison.md
- 误差诊断: outputs/q4_error_diagnosis_report.md
- 天气敏感性检验: outputs/q4_weather_sensitivity_report.md
- 严格递推消融检验: outputs/q4_ablation_report.md
- 低销量鲁棒性策略: outputs/q4_low_volume_strategy_report.md
- 混合策略最终预测说明: outputs/q4_hybrid_final_forecast_report.md
- 候选优化复核: outputs/q4_candidate_model_review.md
- 预测不确定性报告: outputs/forecast_uncertainty_report.md
- 增强误差归因: outputs/q4_error_attribution_enhanced_report.md
- 间歇性需求审查: outputs/intermittent_demand_review.md
- 最终方法选择: outputs/method_search/final_method_selection.md
- 小数预测表: outputs/final_7day_forecast.csv
- 整数预测表: outputs/final_7day_forecast_integer.csv
- 混合策略小数预测表: outputs/final_7day_forecast_hybrid_low_volume.csv
- 混合策略整数预测表: outputs/final_7day_forecast_hybrid_low_volume_integer.csv

A.3 核心 Python 代码

以下列出本次论文最终结论直接依赖的核心新增代码。完整项目代码保存在项目仓库中。复现问题四候选最终结果时，应先运行严格递推消融检验生成 tables/q4_ablation_predictions.csv，再运行低销量策略检验、混合预测生成和候选优化脚本。核心依赖包括 src/features.py、src/models.py、src/evaluation.py 以及 data/processed/modeling_base_table.csv；附录只展示关键阶段脚本，完整可复现代码以项目仓库的 src/ 目录为准。

Listing 1: 严格 7 日递推消融检验代码

```

1 from __future__ import annotations
2
3 import json
4 import sys
5 from dataclasses import dataclass
6 from datetime import date
7 from pathlib import Path
8
9 import matplotlib.pyplot as plt
10 import numpy as np

```

```

11 import pandas as pd
12
13 ROOT = Path(__file__).resolve().parents[1]
14 sys.path.insert(0, str(ROOT))
15
16 from src.evaluation import calculate_metrics
17 from src.features import add_leakage_safe_sales_features,
    build_store_product_panel
18 from src.models import (
19     build_ridge_forecaster,
20     clipped_predict,
21     exp_smoothing_prediction,
22     optimize_exp_smoothing_alpha,
23 )
24
25 TABLES = ROOT / "tables"
26 OUTPUTS = ROOT / "outputs"
27 FIGURES = ROOT / "figures"
28
29 TARGET = "positive_sales"
30 WINDOW_LENGTH = 7
31 VALIDATION_STARTS = pd.to_datetime(
32     ["2022-03-01", "2022-03-08", "2022-03-15", "2022-03-22"]
33 )
34
35 HISTORY_NUMERIC = [
36     "lag_1",
37     "lag_7",
38     "lag_14",
39     "rolling_mean_7",
40     "rolling_mean_14",
41     "rolling_std_7",
42 ]
43 CALENDAR_NUMERIC = ["weekday", "month", "is_weekend"]
44 EVENT_NUMERIC = ["is_holiday", "is_activity_day"]
45 WEATHER_NUMERIC = ["max_temperature", "min_temperature", "wind_power"]
46 ID_CATEGORICAL = ["store_id_str", "product_id_str"]
47 CATEGORY_CATEGORICAL = ["category"]
48 WEATHER_CATEGORICAL = ["weather"]
49

```

```

50 ALL_NUMERIC = HISTORY_NUMERIC + CALENDAR_NUMERIC + EVENT_NUMERIC +
    WEATHER_NUMERIC
51 ALL_CATEGORICAL = ID_CATEGORICAL + CATEGORY_CATEGORICAL + WEATHER_CATEGORICAL
52
53
54 @dataclass(frozen=True)
55 class AblationSpec:
56     variant: str
57     model_label: str
58     feature_scope: str
59     numeric_features: list[str]
60     categorical_features: list[str]
61
62
63 SPECS = [
64     AblationSpec(
65         variant="ridge_full_external",
66         model_label="完整综合Ridge",
67         feature_scope="历史滞后 + 日历 + 节假日/活动日 + 天气 + 门店/商品/类别",
68         numeric_features=ALL_NUMERIC,
69         categorical_features=ALL_CATEGORICAL,
70     ),
71     AblationSpec(
72         variant="ridge_no_weather",
73         model_label="去天气Ridge",
74         feature_scope="去除天气、温度、风力，保留历史、日历、节假日/活动日和门店/
            商品/类别",
75         numeric_features=HISTORY_NUMERIC + CALENDAR_NUMERIC + EVENT_NUMERIC,
76         categorical_features=ID_CATEGORICAL + CATEGORY_CATEGORICAL,
77     ),
78     AblationSpec(
79         variant="ridge_no_activity",
80         model_label="去活动日Ridge",
81         feature_scope="去除活动日，保留历史、日历、节假日、天气和门店/商品/类别",
82         numeric_features=HISTORY_NUMERIC
83         + CALENDAR_NUMERIC
84         + ["is_holiday"]
85         + WEATHER_NUMERIC,
86         categorical_features=ALL_CATEGORICAL,
87     ),

```

```

88 AblationSpec(
89     variant="ridge_no_event",
90     model_label="去节假日活动Ridge",
91     feature_scope="去除节假日和活动日，保留历史、日历、天气和门店/商品/类别",
92     numeric_features=HISTORY_NUMERIC + CALENDAR_NUMERIC + WEATHER_NUMERIC,
93     categorical_features=ALL_CATEGORICAL,
94 ),
95 AblationSpec(
96     variant="ridge_history_id_category",
97     model_label="仅历史与层级Ridge",
98     feature_scope="仅使用历史滞后/滚动特征和门店/商品/类别",
99     numeric_features=HISTORY_NUMERIC,
100    categorical_features=ID_CATEGORICAL + CATEGORY_CATEGORICAL,
101 ),
102 AblationSpec(
103     variant="ridge_history_calendar",
104     model_label="历史日历Ridge",
105     feature_scope="历史滞后/滚动 + 星期/月/周末 + 门店/商品/类别",
106     numeric_features=HISTORY_NUMERIC + CALENDAR_NUMERIC,
107     categorical_features=ID_CATEGORICAL + CATEGORY_CATEGORICAL,
108 ),
109 AblationSpec(
110     variant="ridge_no_history",
111     model_label="无销量历史Ridge",
112     feature_scope="不使用 lag/rolling，仅使用日历、外部变量和门店/商品/类别",
113     numeric_features=CALENDAR_NUMERIC + EVENT_NUMERIC + WEATHER_NUMERIC,
114     categorical_features=ALL_CATEGORICAL,
115 ),
116 ]
117
118
119 def df_to_md(df: pd.DataFrame, max_rows: int | None = None, digits: int = 3)
120     -> str:
121     if df is None or df.empty:
122         return "无。"
123     out = df.copy()
124     if max_rows is not None:
125         out = out.head(max_rows)
126     for col in out.columns:
127         if pd.api.types.is_float_dtype(out[col]):

```

```

127         out[col] = out[col].map(lambda x: "" if pd.isna(x) else f"{x:.{
            digits}f}")
128     cols = list(out.columns)
129     lines = [
130         "| " + " | ".join(cols) + " |",
131         "| " + " | ".join(["---"] * len(cols)) + " |",
132     ]
133     for _, row in out.iterrows():
134         lines.append(
135             "| "
136             + " | ".join("" if pd.isna(row[col]) else str(row[col]) for col in
137                           cols)
138             + " |"
139         )
140     return "\n".join(lines)
141
142 def prepend_result_log(row: str) -> None:
143     path = ROOT / "RESULT_LOG.md"
144     text = path.read_text(encoding="utf-8")
145     if row in text:
146         return
147     lines = text.splitlines()
148     insert_idx = None
149     for idx, line in enumerate(lines):
150         if line.startswith("|-----"):
151             insert_idx = idx + 1
152             break
153     if insert_idx is not None:
154         lines.insert(insert_idx, row)
155         path.write_text("\n".join(lines) + "\n", encoding="utf-8")
156
157
158 def save_csv(df: pd.DataFrame, name: str) -> None:
159     df.to_csv(TABLES / name, index=False, encoding="utf-8-sig")
160     df.to_csv(OUTPUTS / name, index=False, encoding="utf-8-sig")
161
162
163 def prepare_model_frame(panel: pd.DataFrame) -> pd.DataFrame:
164     out = add_leakage_safe_sales_features(panel, TARGET)

```

```

165 out["store_id_str"] = out["store_id"].astype(str)
166 out["product_id_str"] = out["product_id"].astype(str)
167 out["category"] = out["category"].astype(str)
168 out["weather"] = out["weather"].fillna("未知").astype(str)
169 for col in ALL_NUMERIC:
170     out[col] = pd.to_numeric(out[col], errors="coerce").fillna(0.0)
171 out["has_external_data"] = out["has_external_data"].fillna(0).astype(int)
172 return out.sort_values(["store_id", "product_id", "date"]).reset_index(
    drop=True)
173
174
175 def validation_windows(feature_df: pd.DataFrame) -> tuple[list[tuple[pd.
    Timestamp, pd.Timestamp]], list[dict]]:
176     external_dates = set(
177         pd.to_datetime(
178             feature_df.loc[feature_df["has_external_data"] == 1, "date"]
179             ).dt.normalize()
180     )
181     data_dates = set(pd.to_datetime(feature_df["date"]).dt.normalize())
182     windows: list[tuple[pd.Timestamp, pd.Timestamp]] = []
183     excluded: list[dict] = []
184     for start in VALIDATION_STARTS:
185         end = start + pd.Timedelta(days=WINDOW_LENGTH - 1)
186         dates = pd.date_range(start, end, freq="D")
187         has_all_sales = all(d in data_dates for d in dates)
188         has_all_external = all(d in external_dates for d in dates)
189         if has_all_sales and has_all_external:
190             windows.append((start, end))
191         else:
192             excluded.append(
193                 {
194                     "window_start": start.date().isoformat(),
195                     "window_end": end.date().isoformat(),
196                     "reason": "不足完整 7 日销售或外部变量覆盖, 未纳入消融检验",
197                 }
198             )
199     return windows, excluded
200
201
202 def history_feature_row(values: list[float]) -> dict[str, float]:

```

```

203 values = [float(v) for v in values if pd.notna(v)]
204 recent7 = values[-7:] if values else [0.0]
205 recent14 = values[-14:] if values else [0.0]
206 return {
207     "lag_1": float(values[-1]) if len(values) >= 1 else 0.0,
208     "lag_7": float(values[-7]) if len(values) >= 7 else 0.0,
209     "lag_14": float(values[-14]) if len(values) >= 14 else 0.0,
210     "rolling_mean_7": float(np.mean(recent7)),
211     "rolling_mean_14": float(np.mean(recent14)),
212     "rolling_std_7": float(np.std(recent7, ddof=1)) if len(recent7) >= 2
213         else 0.0,
214 }
215
216 def build_lookup_tables(panel: pd.DataFrame, feature_df: pd.DataFrame) ->
217     tuple[pd.DataFrame, dict, dict]:
218     combos = (
219         panel[["store_id", "store_name", "product_id", "product_name", "
220             category"]]
221         .drop_duplicates()
222         .sort_values(["store_id", "product_id"])
223         .reset_index(drop=True)
224     )
225     actual_lookup = (
226         panel.set_index(["date", "store_id", "product_id"])[TARGET].astype(
227             float).to_dict()
228     )
229     external_cols = [
230         "date",
231         "weather",
232         "max_temperature",
233         "min_temperature",
234         "wind_power",
235         "is_holiday",
236         "is_activity_day",
237         "weekday",
238         "month",
239         "is_weekend",
240         "has_external_data",
241     ]

```



```

239     external_lookup = (
240         feature_df[external_cols].drop_duplicates("date").set_index("date").
241         to_dict("index")
242     )
243     return combos, actual_lookup, external_lookup
244
245 def initial_history(panel: pd.DataFrame, combos: pd.DataFrame, window_start:
246     pd.Timestamp) -> dict:
247     before = panel[panel["date"] < window_start]
248     history = {}
249     for row in combos.itertuples():
250         key = (int(row.store_id), int(row.product_id))
251         history[key] = (
252             before[
253                 (before["store_id"] == row.store_id)
254                 & (before["product_id"] == row.product_id)
255             ]
256             .sort_values("date")[TARGET]
257             .astype(float)
258             .tolist()
259         )
260     return history
261
262 def batch_rows_for_date(
263     combos: pd.DataFrame,
264     history: dict,
265     target_date: pd.Timestamp,
266     ext: dict,
267 ) -> pd.DataFrame:
268     rows = []
269     for row in combos.itertuples():
270         rows.append(
271             {
272                 "date": pd.Timestamp(target_date),
273                 "store_id": int(row.store_id),
274                 "store_name": row.store_name,
275                 "product_id": int(row.product_id),
276                 "product_name": row.product_name,

```

```

277         "category": str(row.category),
278         **history_feature_row(history[(int(row.store_id), int(row.
           product_id))]),
279         "weekday": int(ext["weekday"]),
280         "month": int(ext["month"]),
281         "is_weekend": int(ext["is_weekend"]),
282         "is_holiday": int(ext["is_holiday"]),
283         "is_activity_day": int(ext["is_activity_day"]),
284         "max_temperature": float(ext.get("max_temperature", 0.0)),
285         "min_temperature": float(ext.get("min_temperature", 0.0)),
286         "wind_power": float(ext.get("wind_power", 0.0)),
287         "weather": str(ext.get("weather", "未知")),
288         "store_id_str": str(row.store_id),
289         "product_id_str": str(row.product_id),
290     }
291 )
292 return pd.DataFrame(rows)
293
294
295 def append_prediction_rows(
296     rows: list[dict],
297     batch: pd.DataFrame,
298     actual_lookup: dict,
299     variant: str,
300     model_label: str,
301     feature_scope: str,
302     window_start: pd.Timestamp,
303 ) -> None:
304     for pred_row in batch.itertuples():
305         key = (pd.Timestamp(pred_row.date), pred_row.store_id, pred_row.
           product_id)
306         actual = float(actual_lookup.get(key, 0.0))
307         rows.append(
308             {
309                 "validation_mode": "strict_recursive_7day",
310                 "window_start": window_start,
311                 "window_end": window_start + pd.Timedelta(days=WINDOW_LENGTH -
           1),
312                 "date": pd.Timestamp(pred_row.date),
313                 "horizon": int((pd.Timestamp(pred_row.date) - window_start).days

```

```

        + 1),
314     "store_id": pred_row.store_id,
315     "store_name": pred_row.store_name,
316     "product_id": pred_row.product_id,
317     "product_name": pred_row.product_name,
318     "category": pred_row.category,
319     "actual": actual,
320     "prediction": float(pred_row.prediction),
321     "model": variant,
322     "model_label": model_label,
323     "feature_scope": feature_scope,
324     "feature_source": "验证窗口内 lag/rolling 使用前序预测值递推生成",
325 }
326 )
327
328
329 def validate_exp_smoothing(
330     panel: pd.DataFrame,
331     feature_df: pd.DataFrame,
332     windows: list[tuple[pd.Timestamp, pd.Timestamp]],
333 ) -> pd.DataFrame:
334     combos, actual_lookup, external_lookup = build_lookup_tables(panel,
335         feature_df)
336     rows: list[dict] = []
337     variant = "q1_store_product_exp_smoothing"
338     label = "问题一门店-商品简单指数平滑"
339     for window_start, window_end in windows:
340         history = initial_history(panel, combos, window_start)
341         alphas = {
342             (int(row.store_id), int(row.product_id)):
343                 optimize_exp_smoothing_alpha(
344                     pd.Series(history[(int(row.store_id), int(row.product_id))],
345                         dtype=float)
346                 )
347             for row in combos.itertuples()
348         }
349         for target_date in pd.date_range(window_start, window_end, freq="D"):
350             batch_rows = []
351             ext = external_lookup[pd.Timestamp(target_date)]
352             for row in combos.itertuples():

```

```

350     key = (int(row.store_id), int(row.product_id))
351     pred = max(0.0, float(exp_smoothing_prediction(history[key],
352                                                    alphas[key])))
353     batch_rows.append(
354         {
355             "date": pd.Timestamp(target_date),
356             "store_id": int(row.store_id),
357             "store_name": row.store_name,
358             "product_id": int(row.product_id),
359             "product_name": row.product_name,
360             "category": str(row.category),
361             "weather": str(ext.get("weather", "未知")),
362             "prediction": pred,
363         }
364     )
365     batch = pd.DataFrame(batch_rows)
366     append_prediction_rows(
367         rows,
368         batch,
369         actual_lookup,
370         variant,
371         label,
372         "仅使用该门店-商品自身历史销量，不使用外部变量",
373         window_start,
374     )
375     for pred_row in batch.itertuples():
376         history[(int(pred_row.store_id), int(pred_row.product_id))].
377             append(
378                 float(pred_row.prediction)
379             )
380     return pd.DataFrame(rows)
381
382 def validate_ridge_spec(
383     panel: pd.DataFrame,
384     feature_df: pd.DataFrame,
385     windows: list[tuple[pd.Timestamp, pd.Timestamp]],
386     spec: AblationSpec,
387 ) -> pd.DataFrame:
388     combos, actual_lookup, external_lookup = build_lookup_tables(panel,

```

```

        feature_df)
388 rows: list[dict] = []
389 features = spec.numeric_features + spec.categorical_features
390 for window_start, window_end in windows:
391     train = feature_df[
392         (feature_df["date"] < window_start) & (feature_df["
            has_external_data"] == 1)
393     ].copy()
394     model = build_ridge_forecaster(
395         spec.numeric_features,
396         spec.categorical_features,
397         alpha=10.0,
398     )
399     model.fit(train[features], train[TARGET])
400
401     history = initial_history(panel, combos, window_start)
402     for target_date in pd.date_range(window_start, window_end, freq="D"):
403         ext = external_lookup[pd.Timestamp(target_date)]
404         batch = batch_rows_for_date(combos, history, pd.Timestamp(
            target_date), ext)
405         for col in spec.numeric_features:
406             batch[col] = pd.to_numeric(batch[col], errors="coerce").fillna
                (0.0)
407         batch["prediction"] = clipped_predict(model, batch[features])
408         append_prediction_rows(
409             rows,
410             batch,
411             actual_lookup,
412             spec.variant,
413             spec.model_label,
414             spec.feature_scope,
415             window_start,
416         )
417         for pred_row in batch.itertuples():
418             history[(int(pred_row.store_id), int(pred_row.product_id))].
                append(
419                 float(pred_row.prediction)
420             )
421     return pd.DataFrame(rows)
422

```

```

423
424 def aggregate_predictions(preds: pd.DataFrame, level: str) -> pd.DataFrame:
425     if level == "store_product":
426         return preds.copy()
427     if level == "store":
428         group_cols = ["date", "window_start", "model", "model_label", "store_id",
429                        "store_name"]
430     elif level == "product":
431         group_cols = [
432             "date",
433             "window_start",
434             "model",
435             "model_label",
436             "product_id",
437             "product_name",
438             "category",
439         ]
440     elif level == "category":
441         group_cols = ["date", "window_start", "model", "model_label", "category"]
442     else:
443         raise ValueError(level)
444     return (
445         preds.groupby(group_cols, dropna=False, as_index=False)
446         .agg(actual=("actual", "sum"), prediction=("prediction", "sum"))
447         .sort_values(group_cols)
448     )
449
450 def metric_table(preds: pd.DataFrame, windows: list[tuple[pd.Timestamp, pd.
451                  Timestamp]]) -> pd.DataFrame:
452     validation_start = min(start for start, _ in windows).date().isoformat()
453     validation_end = max(end for _, end in windows).date().isoformat()
454     rows = []
455     for level in ["store_product", "store", "product", "category"]:
456         metric_input = aggregate_predictions(preds, level)
457         for (model, label), group in metric_input.groupby(["model", "
458                    model_label"], dropna=False):
459             metrics = calculate_metrics(group["actual"], group["prediction"])
460             feature_scope = (

```

```

459         preds.loc[preds["model"] == model, "feature_scope"].dropna().
            iloc[0]
460     )
461     rows.append(
462         {
463             "validation_mode": "strict_recursive_7day",
464             "validation_start": validation_start,
465             "validation_end": validation_end,
466             "level": level,
467             "model": model,
468             "model_label": label,
469             "feature_scope": feature_scope,
470             **metrics,
471             "WAPE_pct": metrics["WAPE"] * 100,
472             "actual_sum": float(group["actual"].sum()),
473             "prediction_sum": float(group["prediction"].sum()),
474             "bias": float(group["prediction"].sum() - group["actual"].sum()
475                             ()),
476             "bias_pct_of_actual": float(
477                 (group["prediction"].sum() - group["actual"].sum())
478                 / group["actual"].sum()
479             )
480             if group["actual"].sum() != 0
481             else np.nan,
482             "n": int(len(group)),
483         }
484     )
485     out = pd.DataFrame(rows)
486     sp = out[out["level"] == "store_product"].set_index("model")
487     full_wape = float(sp.loc["ridge_full_external", "WAPE_pct"])
488     q1_wape = float(sp.loc["q1_store_product_exp_smoothing", "WAPE_pct"])
489     out["WAPE_pct_point_change_vs_full"] = out["WAPE_pct"] - full_wape
490     out["WAPE_pct_point_change_vs_q1"] = out["WAPE_pct"] - q1_wape
491     return out.sort_values(["level", "WAPE", "MAE", "RMSE"]).reset_index(drop=
492         True)
493
494 def plot_store_product_wape(metrics: pd.DataFrame) -> None:
495     sp = metrics[metrics["level"] == "store_product"].sort_values("WAPE_pct")
496     label_map = {

```

```

496     "q1_store_product_exp_smoothing": "Q1 exp smoothing",
497     "ridge_full_external": "Full Ridge",
498     "ridge_no_weather": "No weather",
499     "ridge_no_activity": "No activity",
500     "ridge_no_event": "No holiday/activity",
501     "ridge_history_id_category": "History + hierarchy",
502     "ridge_history_calendar": "History + calendar",
503     "ridge_no_history": "No sales history",
504 }
505 labels = [label_map.get(model, model) for model in sp["model"]]
506 values = sp["WAPE_pct"].tolist()
507 plt.figure(figsize=(10, 5.8))
508 colors = ["#2F6B55" if m == "q1_store_product_exp_smoothing" else "#6B7280"
509           " for m in sp["model"]]
510 plt.barh(labels, values, color=colors)
511 plt.xlabel("WAPE (%)")
512 plt.title("Q4 strict recursive 7-day ablation at store-product level")
513 for idx, value in enumerate(values):
514     plt.text(value + 0.25, idx, f"{value:.2f}%", va="center", fontsize=9)
515 plt.tight_layout()
516 plt.savefig(FIGURES / "q4_ablation_store_product_wape.png", dpi=180)
517 plt.close()
518
519 def build_report(
520     windows: list[tuple[pd.Timestamp, pd.Timestamp]],
521     excluded: list[dict],
522     metrics: pd.DataFrame,
523 ) -> str:
524     window_df = pd.DataFrame(
525         [
526             {
527                 "window_start": start.date().isoformat(),
528                 "window_end": end.date().isoformat(),
529                 "length": int((end - start).days + 1),
530             }
531             for start, end in windows
532         ]
533     )
534     spec_df = pd.DataFrame(

```



```

535     [
536         {
537             "model": "q1_store_product_exp_smoothing",
538             "model_label": "问题一门店-商品简单指数平滑",
539             "feature_scope": "仅使用该门店-商品自身历史销量, 不使用外部变量",
540         }
541     ]
542     + [
543         {
544             "model": spec.variant,
545             "model_label": spec.model_label,
546             "feature_scope": spec.feature_scope,
547         }
548         for spec in SPECS
549     ]
550 )
551 sp = metrics[metrics["level"] == "store_product"].copy()
552 sp_table = sp[
553     [
554         "model_label",
555         "feature_scope",
556         "MAE",
557         "RMSE",
558         "WAPE_pct",
559         "WAPE_pct_point_change_vs_full",
560         "WAPE_pct_point_change_vs_q1",
561         "actual_sum",
562         "prediction_sum",
563         "bias_pct_of_actual",
564         "n",
565     ]
566 ].copy()
567 level_table = metrics[
568     ["level", "model_label", "MAE", "RMSE", "WAPE_pct", "bias_pct_of_actual",
569     ], "n"]
570 ].copy()
571 best = sp.sort_values(["WAPE", "MAE", "RMSE"]).iloc[0]
572 full = sp[sp["model"] == "ridge_full_external"].iloc[0]
573 no_weather = sp[sp["model"] == "ridge_no_weather"].iloc[0]
574 no_activity = sp[sp["model"] == "ridge_no_activity"].iloc[0]

```

```

574 history_calendar = sp[sp["model"] == "ridge_history_calendar"].iloc[0]
575 no_history = sp[sp["model"] == "ridge_no_history"].iloc[0]
576 q1 = sp[sp["model"] == "q1_store_product_exp_smoothing"].iloc[0]
577
578 weather_gain = no_weather["WAPE_pct"] - full["WAPE_pct"]
579 activity_gain = no_activity["WAPE_pct"] - full["WAPE_pct"]
580 history_penalty = no_history["WAPE_pct"] - full["WAPE_pct"]
581 calendar_gap = history_calendar["WAPE_pct"] - full["WAPE_pct"]
582 ridge_vs_q1 = full["WAPE_pct"] - q1["WAPE_pct"]
583
584 return f"""# 问题四严格 7 日递推消融实验报告
585
586 执行日期: {date.today().isoformat()}
587
588 ## 1. 实验目的
589
590 本实验不继续盲目调参，而是在与问题四主验证一致的严格 7 日递推口径下，检查综合
    Ridge 模型中历史销量、日历、节假日/活动日、天气、门店/商品/类别等信息分别提供
    了多少可复现增益。每个验证窗口开始时只使用窗口开始日前真实销量，窗口内第 2 天
    及以后 `lag` 和 `rolling` 均由前序预测值递推生成。
591
592 ## 2. 验证窗口
593
594 {df_to_md(window_df)}
595
596 未纳入窗口:
597
598 {df_to_md(pd.DataFrame(excluded))}
599
600 ## 3. 消融模型设置
601
602 {df_to_md(spec_df, 20)}
603
604 ## 4. 门店-商品主粒度结果
605
606 {df_to_md(sp_table, 20)}
607
608 完整分层结果见 `tables/q4_ablation_metrics.csv`，逐日预测明细见 `tables/
    q4_ablation_predictions.csv`。图表已保存至 `figures/
    q4_ablation_store_product_wape.png`。

```

5. 关键判断

1. 严格递推门店-商品粒度下，当前最佳模型为 `**{best['model_label']}**`，WAPE 为 `{best['WAPE_pct']:.2f}%`。
2. 完整综合 Ridge 的 WAPE 为 `{full['WAPE_pct']:.2f}%`，相对问题一简单指数平滑高 `{ridge_vs_q1:.2f}` 个百分点。因此论文不能写“综合模型在严格 7 日预测中整体优于问题一模型”。
3. 去天气 Ridge 相比完整 Ridge 的 WAPE 变化为 `{weather_gain:.2f}` 个百分点。天气变量可写作情景补充特征，但不能写成确定因果作用。
4. 去活动日 Ridge 相比完整 Ridge 的 WAPE 变化为 `{activity_gain:.2f}` 个百分点。活动日变量可能包含经营安排内生性，论文中应表述为“与活动日标记相关的销量差异”。
5. 无销量历史 Ridge 相比完整 Ridge 的 WAPE 上升 `{history_penalty:.2f}` 个百分点，说明滞后和滚动销量仍是最核心预测信息。
6. 历史日历 Ridge 相比完整 Ridge 的 WAPE 变化为 `{calendar_gap:.2f}` 个百分点，说明外部变量的边际收益需要结合严格递推和未来可得性谨慎解释。

6. 论文取舍建议

问题四正文建议以“完整综合 Ridge + 严格递推消融”说明综合模型确实整合了前三问信息；但最终模型优劣表述应降调为：综合 Ridge 在门店、商品等聚合层面有较好解释价值，在门店-商品最细粒度上未超过问题一简单指数平滑。消融实验适合放在模型评价或附录，用来证明团队没有简单堆特征，而是做了可复现的方法取舍。

"""

```
def main() -> None:
    for path in [TABLES, OUTPUTS, FIGURES]:
        path.mkdir(parents=True, exist_ok=True)

    raw = pd.read_csv(ROOT / "data" / "processed" / "modeling_base_table.csv")
    raw["date"] = pd.to_datetime(raw["date"])
    panel = build_store_product_panel(raw, TARGET)
    panel["date"] = pd.to_datetime(panel["date"])
    feature_df = prepare_model_frame(panel)

    windows, excluded = validation_windows(feature_df)
    if not windows:
        raise RuntimeError("没有可用于消融实验的完整 7 日验证窗口。")
```

```

638
639 predictions = [validate_exp_smoothing(panel, feature_df, windows)]
640 predictions.extend(validate_ridge_spec(panel, feature_df, windows, spec)
        for spec in SPECS)
641 preds = pd.concat(predictions, ignore_index=True)
642 preds["abs_error"] = (preds["actual"] - preds["prediction"]).abs()
643 save_csv(preds, "q4_ablation_predictions.csv")
644
645 metrics = metric_table(preds, windows)
646 save_csv(metrics, "q4_ablation_metrics.csv")
647 plot_store_product_wape(metrics)
648
649 report = build_report(windows, excluded, metrics)
650 (OUTPUTS / "q4_ablation_report.md").write_text(report, encoding="utf-8")
651
652 sp = metrics[metrics["level"] == "store_product"].set_index("model")
653 best = metrics[metrics["level"] == "store_product"].sort_values("WAPE").
        iloc[0]
654 row = (
655     f"| {date.today().isoformat()} | 问题四严格 7 日递推消融实验 | "
656     "processed: modeling_base_table.csv; 未修改 data/raw | "
657     "问题一指数平滑、完整Ridge、去天气/去活动/去事件/历史日历/无历史等 Ridge
        消融 | "
658     "lag/rolling、门店、商品、类别、日历、节假日、活动日、天气按消融组合控制 |
        "
659     f"门店-商品 WAPE: 最佳={best['model_label']} {best['WAPE_pct']:.2f}%; "
660     f"完整Ridge={sp.loc['ridge_full_external', 'WAPE_pct']:.2f}%; "
661     f"问题一指数平滑={sp.loc['q1_store_product_exp_smoothing', 'WAPE_pct']:.2
        f}%; "
662     f"去天气={sp.loc['ridge_no_weather', 'WAPE_pct']:.2f}% | "
663     "历史销量是核心信息；完整Ridge可作为综合解释模型，但严格递推细粒度未超过问
        题一指数平滑 | "
664     "消融使用同一验证窗口，仍不可把天气/活动日关联写成因果；不据此强行替换最终
        预测文件 | "
665     "将消融表和 WAPE 图写入模型评价或附录 |"
666 )
667 prepend_result_log(row)
668
669 summary = {
670     "validation_mode": "strict_recursive_7day",

```

```

671     "windows": [
672         {"window_start": str(start.date()), "window_end": str(end.date())}
673         for start, end in windows
674     ],
675     "store_product_metrics": metrics[metrics["level"] == "store_product"][
676         ["model", "model_label", "MAE", "RMSE", "WAPE", "WAPE_pct"]
677     ].to_dict("records"),
678     "best_store_product_model": str(best["model"]),
679     "best_store_product_wape_pct": float(best["WAPE_pct"]),
680 }
681 (OUTPUTS / "q4_ablation_summary.json").write_text(
682     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
683 )
684 print(json.dumps(summary, ensure_ascii=False, indent=2))
685
686
687 if __name__ == "__main__":
688     main()

```

Listing 2: 低销量鲁棒性策略检验代码

```

1  from __future__ import annotations
2
3  import json
4  import sys
5  from datetime import date
6  from pathlib import Path
7
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import pandas as pd
11
12 ROOT = Path(__file__).resolve().parents[1]
13 sys.path.insert(0, str(ROOT))
14
15 from src.evaluation import calculate_metrics
16 from src.features import build_store_product_panel
17
18 TABLES = ROOT / "tables"
19 OUTPUTS = ROOT / "outputs"

```

```

20 FIGURES = ROOT / "figures"
21
22 TARGET = "positive_sales"
23 BASE_PREDICTION_PATH = TABLES / "q4_ablation_predictions.csv"
24
25
26 def df_to_md(df: pd.DataFrame, max_rows: int | None = None, digits: int = 3)
    -> str:
27     if df is None or df.empty:
28         return "无。"
29     out = df.copy()
30     if max_rows is not None:
31         out = out.head(max_rows)
32     for col in out.columns:
33         if pd.api.types.is_float_dtype(out[col]):
34             out[col] = out[col].map(lambda x: "" if pd.isna(x) else f"{x:.{
                digits}f}")
35     cols = list(out.columns)
36     lines = [
37         "| " + " | ".join(cols) + " |",
38         "| " + " | ".join(["---" * len(cols)] + " |",
39     ]
40     for _, row in out.iterrows():
41         lines.append(
42             "| "
43             + " | ".join("" if pd.isna(row[col]) else str(row[col]) for col in
                cols)
44             + " |"
45         )
46     return "\n".join(lines)
47
48
49 def prepend_result_log(row: str) -> None:
50     path = ROOT / "RESULT_LOG.md"
51     text = path.read_text(encoding="utf-8")
52     if row in text:
53         return
54     lines = text.splitlines()
55     insert_idx = None
56     for idx, line in enumerate(lines):

```

```

57         if line.startswith("|-----"):
58             insert_idx = idx + 1
59             break
60     if insert_idx is not None:
61         lines.insert(insert_idx, row)
62         path.write_text("\n".join(lines) + "\n", encoding="utf-8")
63
64
65 def save_csv(df: pd.DataFrame, name: str) -> None:
66     df.to_csv(TABLES / name, index=False, encoding="utf-8-sig")
67     df.to_csv(OUTPUTS / name, index=False, encoding="utf-8-sig")
68
69
70 def demand_class_from_history(values: pd.Series) -> str:
71     values = pd.Series(values, dtype=float).fillna(0.0)
72     if values.empty or (values > 0).sum() == 0:
73         return "no_positive_history"
74     avg = float(values.mean())
75     zero_ratio = float((values <= 0).mean())
76     nonzero_ratio = float((values > 0).mean())
77     if nonzero_ratio <= 0.15 or avg < 0.20:
78         return "very_sparse"
79     if zero_ratio >= 0.75 or avg < 0.50:
80         return "low_volume_intermittent"
81     if zero_ratio >= 0.50:
82         return "intermittent"
83     if values.std(ddof=1) / avg >= 1.50:
84         return "volatile"
85     return "regular"
86
87
88 def is_low_volume_class(label: str) -> bool:
89     return label in {"no_positive_history", "very_sparse", "
90         low_volume_intermittent"}
91
92 def classification_at_window(panel: pd.DataFrame, window_start: pd.Timestamp)
93     -> pd.DataFrame:
94     history = panel[panel["date"] < window_start].copy()
95     combo_cols = ["store_id", "store_name", "product_id", "product_name", "

```

```

        category"]
95   combos = (
96       panel[combo_cols]
97       .drop_duplicates()
98       .sort_values(["store_id", "product_id"])
99       .reset_index(drop=True)
100   )
101
102   product_mean = history.groupby("product_id")[TARGET].mean().to_dict()
103   category_mean = history.groupby("category")[TARGET].mean().to_dict()
104   global_mean = float(history[TARGET].mean()) if not history.empty else 0.0
105
106   rows = []
107   for row in combos.itertuples():
108       hist = history[
109           (history["store_id"] == row.store_id)
110           & (history["product_id"] == row.product_id)
111       ].sort_values("date")
112       values = hist[TARGET].astype(float)
113       recent28 = values.tail(28)
114       series_mean = float(values.mean()) if not values.empty else 0.0
115       recent28_mean = float(recent28.mean()) if not recent28.empty else
           series_mean
116       prod_mean = float(product_mean.get(row.product_id, global_mean))
117       cat_mean = float(category_mean.get(row.category, global_mean))
118       hierarchy_target = max(
119           0.0,
120           0.5 * recent28_mean + 0.3 * prod_mean + 0.2 * cat_mean,
121       )
122       label = demand_class_from_history(values)
123       rows.append(
124           {
125               "window_start": pd.Timestamp(window_start),
126               "store_id": int(row.store_id),
127               "store_name": row.store_name,
128               "product_id": int(row.product_id),
129               "product_name": row.product_name,
130               "category": row.category,
131               "history_days": int(len(values)),
132               "history_total_sales": float(values.sum()),

```



```

133         "history_average_sales": series_mean,
134         "history_zero_sales_ratio": float((values <= 0).mean()) if len(
            values) else 1.0,
135         "history_nonzero_sales_ratio": float((values > 0).mean()) if len
            (values) else 0.0,
136         "recent28_mean": recent28_mean,
137         "product_history_mean": prod_mean,
138         "category_history_mean": cat_mean,
139         "hierarchy_shrink_target": hierarchy_target,
140         "demand_class": label,
141         "is_low_volume": int(is_low_volume_class(label)),
142     }
143 )
144 return pd.DataFrame(rows)
145
146
147 def build_classification(panel: pd.DataFrame, window_starts: list[pd.Timestamp
    ]) -> pd.DataFrame:
148     return pd.concat(
149         [classification_at_window(panel, start) for start in window_starts],
150         ignore_index=True,
151     )
152
153
154 def read_base_predictions() -> pd.DataFrame:
155     if not BASE_PREDICTION_PATH.exists():
156         raise FileNotFoundError(
157             "缺少 q4_ablation_predictions.csv, 请先运行 src/stage5_q4_ablation.
                py。"
158         )
159     preds = pd.read_csv(BASE_PREDICTION_PATH)
160     preds["date"] = pd.to_datetime(preds["date"])
161     preds["window_start"] = pd.to_datetime(preds["window_start"])
162     required_models = {"q1_store_product_exp_smoothing", "ridge_full_external"
        }
163     existing = set(preds["model"].unique())
164     missing = sorted(required_models - existing)
165     if missing:
166         raise RuntimeError(f"q4_ablation_predictions.csv 缺少模型: {missing}")
167     return preds

```

```

168
169
170 def pivot_base_predictions(preds: pd.DataFrame) -> pd.DataFrame:
171     key_cols = [
172         "validation_mode",
173         "window_start",
174         "window_end",
175         "date",
176         "horizon",
177         "store_id",
178         "store_name",
179         "product_id",
180         "product_name",
181         "category",
182         "actual",
183     ]
184     subset = preds[preds["model"].isin(["q1_store_product_exp_smoothing", "
        ridge_full_external"])]
185     pivot = (
186         subset.pivot_table(
187             index=key_cols,
188             columns="model",
189             values="prediction",
190             aggfunc="first",
191         )
192         .reset_index()
193         .rename_axis(None, axis=1)
194     )
195     return pivot.rename(
196         columns={
197             "q1_store_product_exp_smoothing": "pred_q1_exp_smoothing",
198             "ridge_full_external": "pred_ridge_full",
199         }
200     )
201
202
203 def build_strategy_predictions(base: pd.DataFrame, cls: pd.DataFrame) -> pd.
    DataFrame:
204     merged = base.merge(
205         cls[

```

```

206         [
207             "window_start",
208             "store_id",
209             "product_id",
210             "history_average_sales",
211             "history_zero_sales_ratio",
212             "recent28_mean",
213             "hierarchy_shrink_target",
214             "demand_class",
215             "is_low_volume",
216         ]
217     ],
218     on=["window_start", "store_id", "product_id"],
219     how="left",
220 )
221 merged["is_low_volume"] = merged["is_low_volume"].fillna(0).astype(int)
222 merged["hierarchy_shrink_target"] = merged["hierarchy_shrink_target"].
    fillna(0.0)
223 merged["q1_low_volume_shrunk"] = np.where(
224     merged["is_low_volume"].eq(1),
225     0.5 * merged["pred_q1_exp_smoothing"] + 0.5 * merged["
        hierarchy_shrink_target"],
226     merged["pred_q1_exp_smoothing"],
227 )
228 merged["ridge_low_volume_shrunk"] = np.where(
229     merged["is_low_volume"].eq(1),
230     0.5 * merged["pred_ridge_full"] + 0.5 * merged["hierarchy_shrink_target
        "],
231     merged["pred_ridge_full"],
232 )
233 merged["hybrid_low_q1_regular_ridge"] = np.where(
234     merged["is_low_volume"].eq(1),
235     merged["pred_q1_exp_smoothing"],
236     merged["pred_ridge_full"],
237 )
238 merged["hybrid_low_shrunk_regular_ridge"] = np.where(
239     merged["is_low_volume"].eq(1),
240     merged["q1_low_volume_shrunk"],
241     merged["pred_ridge_full"],
242 )

```

```

243
244 strategy_specs = [
245     (
246         "q1_store_product_exp_smoothing",
247         "问题一简单指数平滑",
248         "pred_q1_exp_smoothing",
249         "原问题一门店-商品指数平滑，作为严格递推强基准",
250     ),
251     (
252         "ridge_full_external",
253         "完整综合Ridge",
254         "pred_ridge_full",
255         "原问题四完整综合 Ridge，作为综合模型基准",
256     ),
257     (
258         "q1_low_volume_shrunk",
259         "低销量收缩指数平滑",
260         "q1_low_volume_shrunk",
261         "低销量序列用 50% 指数平滑 + 50% 历史层级均值收缩，其余保持指数平滑",
262     ),
263     (
264         "ridge_low_volume_shrunk",
265         "低销量收缩Ridge",
266         "ridge_low_volume_shrunk",
267         "低销量序列用 50% Ridge + 50% 历史层级均值收缩，其余保持 Ridge",
268     ),
269     (
270         "hybrid_low_q1_regular_ridge",
271         "低销量指数平滑-常规Ridge混合",
272         "hybrid_low_q1_regular_ridge",
273         "低销量序列使用指数平滑，非低销量序列使用完整 Ridge",
274     ),
275     (
276         "hybrid_low_shrunk_regular_ridge",
277         "低销量收缩-常规Ridge混合",
278         "hybrid_low_shrunk_regular_ridge",
279         "低销量序列使用收缩指数平滑，非低销量序列使用完整 Ridge",
280     ),
281 ]
282 rows = []

```

```

283     base_cols = [
284         "validation_mode",
285         "window_start",
286         "window_end",
287         "date",
288         "horizon",
289         "store_id",
290         "store_name",
291         "product_id",
292         "product_name",
293         "category",
294         "actual",
295         "history_average_sales",
296         "history_zero_sales_ratio",
297         "recent28_mean",
298         "hierarchy_shrink_target",
299         "demand_class",
300         "is_low_volume",
301     ]
302     for model, label, col, rule in strategy_specs:
303         tmp = merged[base_cols].copy()
304         tmp["prediction"] = np.maximum(0.0, merged[col].astype(float))
305         tmp["model"] = model
306         tmp["model_label"] = label
307         tmp["strategy_rule"] = rule
308         rows.append(tmp)
309     out = pd.concat(rows, ignore_index=True)
310     out["abs_error"] = (out["actual"] - out["prediction"]).abs()
311     return out
312
313
314 def aggregate_predictions(preds: pd.DataFrame, level: str) -> pd.DataFrame:
315     if level == "store_product":
316         return preds.copy()
317     if level == "store":
318         group_cols = ["date", "window_start", "model", "model_label", "store_id",
319                        "store_name"]
320     elif level == "product":
321         group_cols = [

```

```

322         "window_start",
323         "model",
324         "model_label",
325         "product_id",
326         "product_name",
327         "category",
328     ]
329     elif level == "category":
330         group_cols = ["date", "window_start", "model", "model_label", "category"]
331     else:
332         raise ValueError(level)
333     return (
334         preds.groupby(group_cols, dropna=False, as_index=False)
335         .agg(actual=("actual", "sum"), prediction=("prediction", "sum"))
336         .sort_values(group_cols)
337     )
338
339
340 def metric_table(preds: pd.DataFrame) -> pd.DataFrame:
341     rows = []
342     for level in ["store_product", "store", "product", "category"]:
343         metric_input = aggregate_predictions(preds, level)
344         for (model, label), group in metric_input.groupby(["model", "model_label"], dropna=False):
345             metrics = calculate_metrics(group["actual"], group["prediction"])
346             rule = preds.loc[preds["model"] == model, "strategy_rule"].dropna().iloc[0]
347             rows.append(
348                 {
349                     "validation_mode": "strict_recursive_7day",
350                     "level": level,
351                     "model": model,
352                     "model_label": label,
353                     "strategy_rule": rule,
354                     **metrics,
355                     "WAPE_pct": metrics["WAPE"] * 100,
356                     "actual_sum": float(group["actual"].sum()),
357                     "prediction_sum": float(group["prediction"].sum()),
358                     "bias": float(group["prediction"].sum() - group["actual"].sum())

```

```

        ()),
359         "bias_pct_of_actual": float(
360             (group["prediction"].sum() - group["actual"].sum())
361             / group["actual"].sum()
362         )
363         if group["actual"].sum() != 0
364         else np.nan,
365         "n": int(len(group)),
366     }
367 )
368 out = pd.DataFrame(rows)
369 sp = out[out["level"] == "store_product"].set_index("model")
370 q1_wape = float(sp.loc["q1_store_product_exp_smoothing", "WAPE_pct"])
371 ridge_wape = float(sp.loc["ridge_full_external", "WAPE_pct"])
372 out["WAPE_pct_point_change_vs_q1"] = out["WAPE_pct"] - q1_wape
373 out["WAPE_pct_point_change_vs_ridge"] = out["WAPE_pct"] - ridge_wape
374 return out.sort_values(["level", "WAPE", "MAE", "RMSE"]).reset_index(drop=
    True)
375
376
377 def metrics_by_demand_class(preds: pd.DataFrame) -> pd.DataFrame:
378     rows = []
379     for (model, label, cls), group in preds.groupby(
380         ["model", "model_label", "demand_class"], dropna=False
381     ):
382         metrics = calculate_metrics(group["actual"], group["prediction"])
383         rows.append(
384             {
385                 "validation_mode": "strict_recursive_7day",
386                 "model": model,
387                 "model_label": label,
388                 "demand_class": cls,
389                 "is_low_volume": int(group["is_low_volume"].max()),
390                 **metrics,
391                 "WAPE_pct": metrics["WAPE"] * 100,
392                 "actual_sum": float(group["actual"].sum()),
393                 "prediction_sum": float(group["prediction"].sum()),
394                 "n": int(len(group)),
395                 "series_count": int(
396                     group[["window_start", "store_id", "product_id"]].

```

```

drop_duplicates().shape[0]
397     ),
398     }
399 )
400 return pd.DataFrame(rows).sort_values(["demand_class", "WAPE", "model"]).
    reset_index(drop=True)
401
402
403 def classification_summary(cls: pd.DataFrame) -> pd.DataFrame:
404     out = (
405         cls.groupby(["window_start", "demand_class", "is_low_volume"], as_index
406                     =False)
407         .agg(
408             series_count=("product_id", "count"),
409             mean_history_sales=("history_average_sales", "mean"),
410             mean_zero_ratio=("history_zero_sales_ratio", "mean"),
411         )
412         .sort_values(["window_start", "is_low_volume", "demand_class"],
413                     ascending=[True, False, True])
414     )
415     out["window_start"] = out["window_start"].dt.date.astype(str)
416     return out
417
418 def plot_strategy_wape(metrics: pd.DataFrame) -> None:
419     sp = metrics[metrics["level"] == "store_product"].sort_values("WAPE_pct")
420     label_map = {
421         "q1_store_product_exp_smoothing": "Q1 exp smoothing",
422         "ridge_full_external": "Full Ridge",
423         "q1_low_volume_shrunk": "Low-volume shrink SES",
424         "ridge_low_volume_shrunk": "Low-volume shrink Ridge",
425         "hybrid_low_q1_regular_ridge": "Low SES + regular Ridge",
426         "hybrid_low_shrunk_regular_ridge": "Low shrink + regular Ridge",
427     }
428     labels = [label_map.get(model, model) for model in sp["model"]]
429     values = sp["WAPE_pct"].tolist()
430     plt.figure(figsize=(10, 5.4))
431     colors = ["#2F6B55" if model == sp.iloc[0]["model"] else "#6B7280" for
432              model in sp["model"]]
433     plt.barh(labels, values, color=colors)

```



```

432 plt.xlabel("WAPE (%)")
433 plt.title("Strict recursive 7-day low-volume strategy comparison")
434 for idx, value in enumerate(values):
435     plt.text(value + 0.2, idx, f"{value:.2f}%", va="center", fontsize=9)
436 plt.tight_layout()
437 plt.savefig(FIGURES / "q4_low_volume_strategy_wape.png", dpi=180)
438 plt.close()
439
440
441 def build_report(
442     metrics: pd.DataFrame,
443     by_class: pd.DataFrame,
444     cls_summary: pd.DataFrame,
445 ) -> str:
446     sp = metrics[metrics["level"] == "store_product"].copy()
447     sp_table = sp[
448         [
449             "model_label",
450             "strategy_rule",
451             "MAE",
452             "RMSE",
453             "WAPE_pct",
454             "WAPE_pct_point_change_vs_q1",
455             "WAPE_pct_point_change_vs_ridge",
456             "actual_sum",
457             "prediction_sum",
458             "bias_pct_of_actual",
459             "n",
460         ]
461     ]
462     best = sp.sort_values(["WAPE", "MAE", "RMSE"]).iloc[0]
463     q1 = sp[sp["model"] == "q1_store_product_exp_smoothing"].iloc[0]
464     ridge = sp[sp["model"] == "ridge_full_external"].iloc[0]
465
466     low_volume_rows = by_class[by_class["is_low_volume"] == 1][
467         ["model_label", "demand_class", "MAE", "RMSE", "WAPE_pct", "actual_sum",
468          "prediction_sum", "n"]
469     ]
470     class_counts = (
471         cls_summary.groupby(["demand_class", "is_low_volume"], as_index=False)

```

```

471     .agg(
472         mean_series_count=("series_count", "mean"),
473         mean_history_sales=("mean_history_sales", "mean"),
474         mean_zero_ratio=("mean_zero_ratio", "mean"),
475     )
476     .sort_values(["is_low_volume", "demand_class"], ascending=[False, True
477                  ])
478     )
479     best_note = (
480         "可作为论文中的稳健性补充方案，但仍需说明其只是规则化后处理。"
481         if best["model"] not in {"q1_store_product_exp_smoothing", "
482                                ridge_full_external"}
483         else "说明新增低销量后处理没有带来稳定主口径改进，不应强行替换主模型。"
484     )
485
486     return f"""# 问题四低销量序列鲁棒性策略检验报告
487
488 执行日期: {date.today().isoformat()}
489
490 ## 1. 检验目的
491
492 门店-商品序列中低销量和零销量较多，最细粒度 WAPE 偏高。为避免盲目引入 Croston/TSB
493 等复杂间歇需求模型，本检验只评估可解释的低销量分层后处理策略：每个验证窗口开
494 始日前，根据历史平均销量、零销量比例和非零销量比例识别低销量序列；低销量序列可
495 向“近 28 日均值、商品历史均值、类别历史均值”的层级均值收缩。
496
497 分类和收缩目标均只使用窗口开始日前历史数据，未使用验证窗口真实销量。
498
499 ## 2. 低销量分类概况
500
501 {df_to_md(class_counts, 20)}
502
503 逐窗口分类明细见 `tables/q4_low_volume_classification.csv`。
504
505 ## 3. 门店-商品主粒度策略比较
506
507 {df_to_md(sp_table, 20)}
508
509 完整分层指标见 `tables/q4_low_volume_strategy_metrics.csv`，按需求类别拆分指标
510 见 `tables/q4_low_volume_strategy_by_class.csv`。图表已保存至 `figures/

```

```

    q4_low_volume_strategy_wape.png`。
505
506 ## 4. 低销量类别误差
507
508 {df_to_md(low_volume_rows, 30)}
509
510 ## 5. 方法取舍
511
512 严格递推门店-商品粒度下，本检验最佳策略为 **{best['model_label']}**，WAPE={best
    ['WAPE_pct']:.2f}%。问题一简单指数平滑 WAPE={q1['WAPE_pct']:.2f}%, 完整综合
    Ridge WAPE={ridge['WAPE_pct']:.2f}%。
513
514 {best_note}
515
516 论文建议把低销量分析写入模型评价和局限性部分：低销量序列是细粒度误差的主要来源之
    一；简单的层级收缩可作为稳健性检查，但若未显著优于主模型，就不应为了“看起来高
    级”而替换最终预测模型。
517 """
518
519
520 def main() -> None:
521     for path in [TABLES, OUTPUTS, FIGURES]:
522         path.mkdir(parents=True, exist_ok=True)
523
524     raw = pd.read_csv(ROOT / "data" / "processed" / "modeling_base_table.csv")
525     raw["date"] = pd.to_datetime(raw["date"])
526     panel = build_store_product_panel(raw, TARGET)
527     panel["date"] = pd.to_datetime(panel["date"])
528
529     base_preds = read_base_predictions()
530     base = pivot_base_predictions(base_preds)
531     window_starts = sorted(pd.to_datetime(base["window_start"]).
        drop_duplicates().tolist())
532     cls = build_classification(panel, window_starts)
533     save_csv(cls, "q4_low_volume_classification.csv")
534
535     strategy_preds = build_strategy_predictions(base, cls)
536     save_csv(strategy_preds, "q4_low_volume_strategy_predictions.csv")
537
538     metrics = metric_table(strategy_preds)

```

```

539 by_class = metrics_by_demand_class(strategy_preds)
540 cls_summary = classification_summary(cls)
541 save_csv(metrics, "q4_low_volume_strategy_metrics.csv")
542 save_csv(by_class, "q4_low_volume_strategy_by_class.csv")
543 save_csv(cls_summary, "q4_low_volume_classification_summary.csv")
544 plot_strategy_wape(metrics)
545
546 report = build_report(metrics, by_class, cls_summary)
547 (OUTPUTS / "q4_low_volume_strategy_report.md").write_text(report, encoding
    ="utf-8")
548
549 sp = metrics[metrics["level"] == "store_product"].set_index("model")
550 best = metrics[metrics["level"] == "store_product"].sort_values("WAPE").
    iloc[0]
551 low_share = float(cls["is_low_volume"].mean())
552 row = (
553     f"| {date.today().isoformat()} | 问题四低销量序列鲁棒性策略检验 | "
554     "processed: modeling_base_table.csv; 输入 q4_ablation_predictions.csv;
        未修改 data/raw | "
555     "问题一指数平滑、完整Ridge、低销量层级收缩、低销量/常规混合策略 | "
556     "窗口开始日前历史均值、零销量比例、近28日均值、商品均值、类别均值; 验证窗
        口内基模型仍为严格递推预测 | "
557     f"低销量序列窗口占比={low_share:.1%}; 门店-商品 WAPE: 最佳={best['
        model_label']} {best['WAPE_pct']:.2f}%; "
558     f"问题一指数平滑={sp.loc['q1_store_product_exp_smoothing', 'WAPE_pct']:.2
        f}%; "
559     f"完整Ridge={sp.loc['ridge_full_external', 'WAPE_pct']:.2f}% | "
560     "低销量策略用于解释和稳健性检查; 是否替换主模型以严格递推指标为准 | "
561     "分类与收缩只使用窗口开始日前历史数据; 层级收缩为规则后处理, 不等同于新因
        果机制 | "
562     "将低销量误差来源和策略取舍写入模型评价/附录 | "
563 )
564 prepend_result_log(row)
565
566 summary = {
567     "validation_mode": "strict_recursive_7day",
568     "low_volume_window_share": low_share,
569     "store_product_metrics": metrics[metrics["level"] == "store_product"][
570         ["model", "model_label", "MAE", "RMSE", "WAPE", "WAPE_pct"]
571     ].to_dict("records"),

```

```

572     "best_store_product_model": str(best["model"]),
573     "best_store_product_wape_pct": float(best["WAPE_pct"]),
574 }
575 (OUTPUTS / "q4_low_volume_strategy_summary.json").write_text(
576     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
577 )
578 print(json.dumps(summary, ensure_ascii=False, indent=2))
579
580
581 if __name__ == "__main__":
582     main()

```

Listing 3: 低销量混合策略最终预测代码

```

1  from __future__ import annotations
2
3  import json
4  import sys
5  from datetime import date
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10
11 ROOT = Path(__file__).resolve().parents[1]
12 sys.path.insert(0, str(ROOT))
13
14 from src.features import build_store_product_panel
15 from src.models import exp_smoothing_prediction, optimize_exp_smoothing_alpha
16 from src.stage5_q4_low_volume_strategy import classification_at_window
17
18 TABLES = ROOT / "tables"
19 OUTPUTS = ROOT / "outputs"
20
21 TARGET = "positive_sales"
22 FUTURE_START = pd.Timestamp("2022-04-01")
23
24
25 def df_to_md(df: pd.DataFrame, max_rows: int | None = None, digits: int = 3)
    -> str:

```

```

26     if df is None or df.empty:
27         return "无。"
28     out = df.copy()
29     if max_rows is not None:
30         out = out.head(max_rows)
31     for col in out.columns:
32         if pd.api.types.is_float_dtype(out[col]):
33             out[col] = out[col].map(lambda x: "" if pd.isna(x) else f"{x:.{
34                 digits}f}")
35     cols = list(out.columns)
36     lines = [
37         "| " + " | ".join(cols) + " |",
38         "| " + " | ".join(["---"] * len(cols)) + " |",
39     ]
40     for _, row in out.iterrows():
41         lines.append(
42             "| "
43             + " | ".join("" if pd.isna(row[col]) else str(row[col]) for col in
44                 cols)
45             + " |"
46         )
47     return "\n".join(lines)
48
49 def prepend_result_log(row: str) -> None:
50     path = ROOT / "RESULT_LOG.md"
51     text = path.read_text(encoding="utf-8")
52     if row in text:
53         return
54     lines = text.splitlines()
55     insert_idx = None
56     for idx, line in enumerate(lines):
57         if line.startswith("|-----"):
58             insert_idx = idx + 1
59             break
60     if insert_idx is not None:
61         lines.insert(insert_idx, row)
62         path.write_text("\n".join(lines) + "\n", encoding="utf-8")
63

```

```

64 def save_csv(df: pd.DataFrame, name: str) -> None:
65     df.to_csv(TABLES / name, index=False, encoding="utf-8-sig")
66     df.to_csv(OUTPUTS / name, index=False, encoding="utf-8-sig")
67
68
69 def build_q1_future_predictions(panel: pd.DataFrame, future_template: pd.
    DataFrame) -> pd.DataFrame:
70     combo_cols = ["store_id", "store_name", "product_id", "product_name", "
        category"]
71     combos = (
72         panel[combo_cols]
73         .drop_duplicates()
74         .sort_values(["store_id", "product_id"])
75         .reset_index(drop=True)
76     )
77     history = {}
78     alphas = {}
79     for row in combos.itertuples():
80         key = (int(row.store_id), int(row.product_id))
81         values = (
82             panel[
83                 (panel["store_id"] == row.store_id)
84                 & (panel["product_id"] == row.product_id)
85             ]
86             .sort_values("date")[TARGET]
87             .astype(float)
88             .tolist()
89         )
90         history[key] = values
91         alphas[key] = optimize_exp_smoothing_alpha(pd.Series(values, dtype=
            float))
92
93     ext_cols = [
94         "date",
95         "weather",
96         "is_holiday",
97         "is_weekend",
98         "is_activity_day",
99         "external_scenario_note",
100    ]

```

```

101 future_ext = future_template[ext_cols].drop_duplicates("date").sort_values
    ("date")
102 rows = []
103 for ext in future_ext.itertuples():
104     for combo in combos.itertuples():
105         key = (int(combo.store_id), int(combo.product_id))
106         pred = max(0.0, float(exp_smoothing_prediction(history[key], alphas
            [key])))
107         rows.append(
108             {
109                 "date": pd.Timestamp(ext.date),
110                 "store_id": int(combo.store_id),
111                 "store_name": combo.store_name,
112                 "product_id": int(combo.product_id),
113                 "product_name": combo.product_name,
114                 "category": combo.category,
115                 "weather": ext.weather,
116                 "is_holiday": int(ext.is_holiday),
117                 "is_weekend": int(ext.is_weekend),
118                 "is_activity_day": int(ext.is_activity_day),
119                 "pred_q1_exp_smoothing": pred,
120                 "external_scenario_note": ext.external_scenario_note,
121             }
122         )
123         history[key].append(pred)
124     return pd.DataFrame(rows)
125
126
127 def integerize_forecast(df: pd.DataFrame) -> pd.DataFrame:
128     out = df.copy()
129     out["predicted_sales"] = np.rint(np.maximum(0.0, out["predicted_sales"].
        astype(float))).astype(int)
130     return out
131
132
133 def write_report(
134     future: pd.DataFrame,
135     store_total: pd.DataFrame,
136     category_total: pd.DataFrame,
137     cls: pd.DataFrame,

```



```

138 ) -> None:
139     strategy_metrics = pd.read_csv(TABLES / "q4_low_volume_strategy_metrics.
        csv")
140     sp = strategy_metrics[strategy_metrics["level"] == "store_product"].
        set_index("model")
141     low_count = int(cls["is_low_volume"].sum())
142     total_count = int(len(cls))
143     report = f"""\# 问题四低销量混合策略最终预测说明
144
145 执行日期: {date.today().isoformat()}
146
147 ## 1. 生成目的
148
149 原最终预测文件 `outputs/final_7day_forecast.csv` 保留为完整综合 Ridge 条件预测。
    本文件新增一个不覆盖原结果的候选最终预测: `outputs/
    final_7day_forecast_hybrid_low_volume.csv`。该策略来自严格 7 日递推低销量鲁
    棒性检验: 低销量门店--商品序列使用问题一简单指数平滑, 非低销量序列使用完整综合
    Ridge。
150
151 ## 2. 验证依据
152
153 在 2022-03-01 至 2022-03-28 的 4 个完整严格 7 日递推窗口上, 低销量指数平滑--常规
    Ridge 混合策略门店--商品 WAPE 为 {sp.loc['hybrid_low_q1_regular_ridge', '
    WAPE_pct']:.2f}%, 问题一简单指数平滑为 {sp.loc['
    q1_store_product_exp_smoothing', 'WAPE_pct']:.2f}%, 完整综合 Ridge 为 {sp.
    loc['ridge_full_external', 'WAPE_pct']:.2f}%。
154
155 ## 3. 未来预测规则
156
157 以 2022-04-01 作为预测窗口起点, 只使用 2022-03-31 及以前历史销量判定低销量序列。
    本次共有 {low_count}/{total_count} 条门店--商品组合被判为低销量。低销量组合使
    用简单指数平滑递推预测; 其余组合沿用完整综合 Ridge 预测。未来天气、温度、风力
    和活动日仍沿用历史同期情景, 因此预测仍是条件预测。
158
159 ## 4. 预测汇总
160
161 未来 7 天混合策略预测总销量为 {future['predicted_sales'].sum():.3f}。
162
163 门店汇总:
164

```

```

165 {df_to_md(store_total, 20)}
166
167 类别汇总:
168
169 {df_to_md(category_total, 20)}
170
171 ## 5. 论文表述边界
172
173 该混合策略可以作为最终预测候选，因为它在同一严格递推验证口径下略优于原强基准和完整
    Ridge；但它是基于低销量诊断的规则化模型选择，不应被写成复杂机器学习模型带来显
    著突破。正式论文可表述为“在综合 Ridge 的基础上，对低销量稀疏序列采用简单指数
    平滑兜底，形成稳健混合预测策略”。
174 """
175 (OUTPUTS / "q4_hybrid_final_forecast_report.md").write_text(report,
    encoding="utf-8")
176
177
178 def main() -> None:
179     for path in [TABLES, OUTPUTS]:
180         path.mkdir(parents=True, exist_ok=True)
181
182     ridge_path = OUTPUTS / "final_7day_forecast.csv"
183     if not ridge_path.exists():
184         raise FileNotFoundError("缺少 outputs/final_7day_forecast.csv。")
185     ridge = pd.read_csv(ridge_path)
186     ridge["date"] = pd.to_datetime(ridge["date"])
187     ridge = ridge.rename(columns={"predicted_sales": "pred_ridge_full"})
188
189     raw = pd.read_csv(ROOT / "data" / "processed" / "modeling_base_table.csv")
190     raw["date"] = pd.to_datetime(raw["date"])
191     panel = build_store_product_panel(raw, TARGET)
192     panel["date"] = pd.to_datetime(panel["date"])
193
194     q1 = build_q1_future_predictions(panel, ridge)
195     cls = classification_at_window(panel, FUTURE_START)
196     save_csv(cls, "q4_hybrid_future_low_volume_classification.csv")
197
198     merged = ridge.merge(
199         q1[
200             [

```

```

201         "date",
202         "store_id",
203         "product_id",
204         "pred_q1_exp_smoothing",
205     ]
206 ],
207 on=["date", "store_id", "product_id"],
208 how="left",
209 ).merge(
210     cls[["store_id", "product_id", "demand_class", "is_low_volume"]],
211     on=["store_id", "product_id"],
212     how="left",
213 )
214 merged["is_low_volume"] = merged["is_low_volume"].fillna(0).astype(int)
215 merged["predicted_sales"] = np.where(
216     merged["is_low_volume"].eq(1),
217     merged["pred_q1_exp_smoothing"],
218     merged["pred_ridge_full"],
219 )
220 merged["predicted_sales"] = np.where(
221     np.abs(merged["predicted_sales"].astype(float)) < 1e-6,
222     0.0,
223     merged["predicted_sales"].astype(float),
224 )
225 merged["model"] = "hybrid_low_q1_regular_ridge"
226 merged["model_label"] = "低销量指数平滑-常规Ridge混合"
227 merged["external_scenario_note"] = (
228     merged["external_scenario_note"]
229     + "; 低销量组合使用问题一指数平滑, 非低销量组合使用综合Ridge"
230 )
231 out_cols = [
232     "date",
233     "store_id",
234     "store_name",
235     "product_id",
236     "product_name",
237     "category",
238     "weather",
239     "is_holiday",
240     "is_weekend",

```

```

241     "is_activity_day",
242     "predicted_sales",
243     "model",
244     "model_label",
245     "demand_class",
246     "is_low_volume",
247     "external_scenario_note",
248 ]
249 final = merged[out_cols].sort_values(["date", "store_id", "product_id"]).
    reset_index(drop=True)
250 final["date"] = final["date"].dt.date.astype(str)
251 save_csv(final, "final_7day_forecast_hybrid_low_volume.csv")
252 save_csv(integerize_forecast(final), "
    final_7day_forecast_hybrid_low_volume_integer.csv")
253
254 store_total = (
255     final.groupby(["store_id", "store_name"], as_index=False)["
        predicted_sales"]
256     .sum()
257     .rename(columns={"predicted_sales": "predicted_7day_sales"})
258     .sort_values("predicted_7day_sales", ascending=False)
259 )
260 category_total = (
261     final.groupby("category", as_index=False)["predicted_sales"]
262     .sum()
263     .rename(columns={"predicted_sales": "predicted_7day_sales"})
264     .sort_values("predicted_7day_sales", ascending=False)
265 )
266 product_total = (
267     final.groupby(["product_id", "product_name", "category"], as_index=
        False)["predicted_sales"]
268     .sum()
269     .rename(columns={"predicted_sales": "predicted_7day_sales"})
270     .sort_values("predicted_7day_sales", ascending=False)
271 )
272 save_csv(store_total, "q4_hybrid_forecast_7day_total_by_store.csv")
273 save_csv(category_total, "q4_hybrid_forecast_7day_total_by_category.csv")
274 save_csv(product_total, "q4_hybrid_forecast_7day_total_by_product.csv")
275 write_report(final, store_total, category_total, cls)
276

```

```

277 integer_total = int(integerize_forecast(final)["predicted_sales"].sum())
278 summary = {
279     "model": "hybrid_low_q1_regular_ridge",
280     "future_rows": int(len(final)),
281     "low_volume_series": int(cls["is_low_volume"].sum()),
282     "total_series": int(len(cls)),
283     "future_total_sales": float(final["predicted_sales"].sum()),
284     "future_integer_total_sales": integer_total,
285     "original_ridge_total_sales": float(ridge["pred_ridge_full"].sum()),
286 }
287 (OUTPUTS / "q4_hybrid_final_forecast_summary.json").write_text(
288     json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
289 )
290
291 row = (
292     f"| {date.today().isoformat()} | 问题四低销量混合策略最终预测 | "
293     "processed: modeling_base_table.csv; 输入原 final_7day_forecast.csv; 未
294     覆盖原预测表 | "
295     "低销量指数平滑 + 常规序列综合 Ridge 混合 | "
296     "2022-04-01 前历史销量判定低销量; 低销量组合用指数平滑, 非低销量组合沿用综
297     合 Ridge; 外部变量仍为历史同期情景 | "
298     f"未来7天混合预测总量={summary['future_total_sales']:.3f}; 整数化总量={
299     summary['future_integer_total_sales']}; "
300     f"低销量组合={summary['low_volume_series']}/{summary['total_series']} |
301     "
302     "新增不覆盖原结果的候选最终预测表; 严格递推验证依据来自
303     q4_low_volume_strategy_metrics.csv | "
304     "混合策略为规则化稳健处理, 不应写成复杂模型显著突破; 未来外部变量仍为条件
305     情景 | "
306     "论文问题四和模型评价中说明最终预测表取舍 |"
307 )
308
309 prepend_result_log(row)
310 print(json.dumps(summary, ensure_ascii=False, indent=2))
311
312 if __name__ == "__main__":
313     main()

```

Listing 4: 候选优化与不确定性补充代码

```

1 from __future__ import annotations
2
3 import json
4 import sys
5 from datetime import date
6 from pathlib import Path
7 from typing import Callable
8
9 import matplotlib.pyplot as plt
10 import numpy as np
11 import pandas as pd
12
13 ROOT = Path(__file__).resolve().parents[1]
14 sys.path.insert(0, str(ROOT))
15
16 from src.evaluation import calculate_metrics
17
18 TABLES = ROOT / "tables"
19 OUTPUTS = ROOT / "outputs"
20 FIGURES = ROOT / "figures"
21
22 CURRENT_MODEL = "hybrid_current_q1_regular_ridge"
23 Q1_MODEL = "q1_store_product_exp_smoothing"
24 RIDGE_MODEL = "ridge_full_external"
25 IMPROVEMENT_THRESHOLD_PCT = 0.30
26
27
28 def df_to_md(df: pd.DataFrame, max_rows: int | None = None, digits: int = 3)
    -> str:
29     if df is None or df.empty:
30         return "无。"
31     out = df.copy()
32     if max_rows is not None:
33         out = out.head(max_rows)
34     for col in out.columns:
35         if pd.api.types.is_float_dtype(out[col]):
36             out[col] = out[col].map(lambda x: "" if pd.isna(x) else f"{x:.{
                digits}f}")
37     cols = list(out.columns)
38     lines = [

```

```

39         "| " + " | ".join(cols) + " |",
40         "| " + " | ".join(["---"] * len(cols)) + " |",
41     ]
42     for _, row in out.iterrows():
43         lines.append(
44             "| "
45             + " | ".join("" if pd.isna(row[col]) else str(row[col]) for col in
46                           cols)
47             + " |"
48         )
49     return "\n".join(lines)
50
51 def prepend_result_log(row: str) -> None:
52     path = ROOT / "RESULT_LOG.md"
53     text = path.read_text(encoding="utf-8")
54     if row in text:
55         return
56     lines = text.splitlines()
57     insert_idx = None
58     for idx, line in enumerate(lines):
59         if line.startswith("|-----"):
60             insert_idx = idx + 1
61             break
62     if insert_idx is not None:
63         lines.insert(insert_idx, row)
64         path.write_text("\n".join(lines) + "\n", encoding="utf-8")
65
66
67 def save_csv(df: pd.DataFrame, name: str) -> None:
68     df.to_csv(TABLES / name, index=False, encoding="utf-8-sig")
69     df.to_csv(OUTPUTS / name, index=False, encoding="utf-8-sig")
70
71
72 def setup_plot_style() -> None:
73     plt.rcParams["font.sans-serif"] = [
74         "Microsoft YaHei",
75         "SimHei",
76         "Arial Unicode MS",
77         "DejaVu Sans",

```

```

78 ]
79 plt.rcParams["axes.unicode_minus"] = False
80 plt.rcParams["figure.dpi"] = 150
81
82
83 def load_strategy_base() -> pd.DataFrame:
84     strategy_path = TABLES / "q4_low_volume_strategy_predictions.csv"
85     cls_path = TABLES / "q4_low_volume_classification.csv"
86     if not strategy_path.exists():
87         raise FileNotFoundError("缺少 tables/q4_low_volume_strategy_predictions
            .csv。")
88     if not cls_path.exists():
89         raise FileNotFoundError("缺少 tables/q4_low_volume_classification.csv。
            ")
90
91     preds = pd.read_csv(strategy_path)
92     cls = pd.read_csv(cls_path)
93     for frame in [preds, cls]:
94         frame["window_start"] = pd.to_datetime(frame["window_start"])
95     preds["date"] = pd.to_datetime(preds["date"])
96
97     key_cols = [
98         "validation_mode",
99         "window_start",
100         "window_end",
101         "date",
102         "horizon",
103         "store_id",
104         "store_name",
105         "product_id",
106         "product_name",
107         "category",
108         "actual",
109     ]
110     model_map = {
111         "q1_store_product_exp_smoothing": "pred_q1_exp_smoothing",
112         "ridge_full_external": "pred_ridge_full",
113     }
114     pivot = (
115         preds[preds["model"].isin(model_map)]

```



```

116         .pivot_table(index=key_cols, columns="model", values="prediction",
117                        aggfunc="first")
118         .reset_index()
119         .rename_axis(None, axis=1)
120         .rename(columns=model_map)
121     )
122     required = {"pred_q1_exp_smoothing", "pred_ridge_full"}
123     missing = required - set(pivot.columns)
124     if missing:
125         raise ValueError(f"低销量策略预测表缺少必要基准预测列: {sorted(missing)}")
126
127     cls_cols = [
128         "window_start",
129         "store_id",
130         "product_id",
131         "history_average_sales",
132         "history_zero_sales_ratio",
133         "recent28_mean",
134         "product_history_mean",
135         "category_history_mean",
136         "hierarchy_shrink_target",
137         "demand_class",
138         "is_low_volume",
139     ]
140     out = pivot.merge(cls[cls_cols], on=["window_start", "store_id", "product_id"], how="left")
141     out["is_low_volume"] = out["is_low_volume"].fillna(0).astype(int)
142     numeric_cols = [
143         "actual",
144         "pred_q1_exp_smoothing",
145         "pred_ridge_full",
146         "history_average_sales",
147         "history_zero_sales_ratio",
148         "recent28_mean",
149         "product_history_mean",
150         "category_history_mean",
151         "hierarchy_shrink_target",
152     ]
153     for col in numeric_cols:

```

```

153     out[col] = pd.to_numeric(out[col], errors="coerce").fillna(0.0)
154 out["window_end"] = pd.to_datetime(out["window_end"])
155 return out.sort_values(["window_start", "date", "store_id", "product_id"])
156     .reset_index(
157         drop=True
158     )
159
160 def candidate_thresholds() -> list[tuple[str, str, str, Callable[[pd.DataFrame
161     ], pd.Series]]]:
162     return [
163         (
164             "current_rule",
165             "当前低销量规则",
166             "沿用 no_positive_history、very_sparse、low_volume_intermittent 三类
167             低销量判定",
168             lambda df: df["is_low_volume"].eq(1),
169         ),
170         (
171             "avg_lt_0_2",
172             "历史均值 <0.2",
173             "窗口开始日前门店-商品历史日均销量小于 0.2",
174             lambda df: df["history_average_sales"].lt(0.2),
175         ),
176         (
177             "avg_lt_0_3",
178             "历史均值 <0.3",
179             "窗口开始日前门店-商品历史日均销量小于 0.3",
180             lambda df: df["history_average_sales"].lt(0.3),
181         ),
182         (
183             "zero_ge_0_6",
184             "零销量比例 >=0.6",
185             "窗口开始日前零销量天数占比不低于 60%",
186             lambda df: df["history_zero_sales_ratio"].ge(0.6),
187         ),
188         (
189             "zero_ge_0_75",
190             "零销量比例 >=0.75",
191             "窗口开始日前零销量天数占比不低于 75%",

```

```

190         lambda df: df["history_zero_sales_ratio"].ge(0.75),
191     ),
192     (
193         "avg_lt_0_3_or_zero_ge_0_75",
194         "历史均值 <0.3 或零销量比例 >=0.75",
195         "低均值和高零销量比例的并集判定",
196         lambda df: df["history_average_sales"].lt(0.3)
197         | df["history_zero_sales_ratio"].ge(0.75),
198     ),
199 ]
200
201
202 def low_volume_predictors() -> list[tuple[str, str, str, Callable[[pd.
    DataFrame], pd.Series]]]:
203     return [
204         (
205             "q1_exp_smoothing",
206             "低销量用指数平滑",
207             "低销量序列使用问题一门店-商品指数平滑，常规序列使用完整 Ridge",
208             lambda df: df["pred_q1_exp_smoothing"],
209         ),
210         (
211             "recent28_mean",
212             "低销量用近28日均值",
213             "低销量序列使用窗口开始日前近 28 日均值，常规序列使用完整 Ridge",
214             lambda df: df["recent28_mean"],
215         ),
216         (
217             "product_mean",
218             "低销量用商品均值",
219             "低销量序列使用窗口开始日前同商品历史均值，常规序列使用完整 Ridge",
220             lambda df: df["product_history_mean"],
221         ),
222         (
223             "category_mean",
224             "低销量用类别均值",
225             "低销量序列使用窗口开始日前同类别历史均值，常规序列使用完整 Ridge",
226             lambda df: df["category_history_mean"],
227         ),
228         (

```

```

229         "hierarchy_shrink",
230         "低销量用层级收缩均值",
231         "低销量序列使用近28日、商品均值、类别均值的层级收缩目标，常规序列使用完
            整 Ridge",
232         lambda df: df["hierarchy_shrink_target"],
233     ),
234     (
235         "q1_hierarchy_blend",
236         "低销量指数平滑+层级收缩",
237         "低销量序列使用 70% 指数平滑 + 30% 层级收缩目标，常规序列使用完整
            Ridge",
238         lambda df: 0.7 * df["pred_q1_exp_smoothing"]
239         + 0.3 * df["hierarchy_shrink_target"],
240     ),
241 ]
242
243
244 def base_columns() -> list[str]:
245     return [
246         "validation_mode",
247         "window_start",
248         "window_end",
249         "date",
250         "horizon",
251         "store_id",
252         "store_name",
253         "product_id",
254         "product_name",
255         "category",
256         "actual",
257         "history_average_sales",
258         "history_zero_sales_ratio",
259         "demand_class",
260         "is_low_volume",
261     ]
262
263
264 def build_candidate_predictions(base: pd.DataFrame) -> pd.DataFrame:
265     rows = []
266

```

```

267 def append_model(
268     model: str,
269     label: str,
270     rule: str,
271     prediction: pd.Series | np.ndarray,
272     low_mask: pd.Series | None = None,
273     threshold_label: str = "不适用",
274 ) -> None:
275     tmp = base[base_columns()].copy()
276     tmp["prediction"] = np.maximum(0.0, pd.Series(prediction, index=base.
277         index).astype(float))
278     tmp["model"] = model
279     tmp["model_label"] = label
280     tmp["strategy_rule"] = rule
281     tmp["threshold_rule"] = threshold_label
282     tmp["low_volume_selected"] = low_mask.astype(int).to_numpy() if
283         low_mask is not None else 0
284     rows.append(tmp)
285
286 append_model(
287     Q1_MODEL,
288     "问题一简单指数平滑",
289     "所有门店-商品序列均使用问题一指数平滑",
290     base["pred_q1_exp_smoothing"],
291 )
292
293 append_model(
294     RIDGE_MODEL,
295     "完整综合Ridge",
296     "所有门店-商品序列均使用完整综合 Ridge",
297     base["pred_ridge_full"],
298 )
299
300 for threshold_id, threshold_label, threshold_rule, mask_fn in
301     candidate_thresholds():
302     low_mask = mask_fn(base).fillna(False)
303     for pred_id, pred_label, pred_rule, pred_fn in low_volume_predictors():
304         pred = np.where(low_mask, pred_fn(base), base["pred_ridge_full"])
305         model = f"hybrid_{threshold_id}_{pred_id}"
306         label = f"{threshold_label}-{pred_label}"
307         if threshold_id == "current_rule" and pred_id == "q1_exp_smoothing"

```

```

        :
304         model = CURRENT_MODEL
305         label = "当前混合策略：低销量指数平滑-常规Ridge"
306         append_model(
307             model=model,
308             label=label,
309             rule=pred_rule,
310             prediction=pred,
311             low_mask=low_mask,
312             threshold_label=threshold_label,
313         )
314
315     candidates = pd.concat(rows, ignore_index=True)
316     candidates["abs_error"] = (candidates["actual"] - candidates["prediction"
317                               ]).abs()
318     return candidates
319
320 def apply_q1_aggregate_calibration(
321     base: pd.DataFrame,
322     candidate_preds: pd.DataFrame,
323     level: str,
324 ) -> pd.DataFrame:
325     group_map = {
326         "overall": [],
327         "store": ["store_id"],
328         "product": ["product_id"],
329         "category": ["category"],
330     }
331     group_cols = group_map[level]
332     current = candidate_preds[candidate_preds["model"] == CURRENT_MODEL].copy()
333     if current.empty:
334         raise ValueError("候选预测中找不到当前混合策略。")
335     current = current.drop(columns=["prediction"])
336     current = current.merge(
337         base[
338             [
339                 "window_start",
340                 "date",

```

```

341         "store_id",
342         "product_id",
343         "category",
344         "pred_q1_exp_smoothing",
345     ]
346 ],
347 on=["window_start", "date", "store_id", "product_id", "category"],
348 how="left",
349 )
350 current = current.rename(columns={"pred_q1_exp_smoothing": "
    q1_target_prediction"})
351 original = candidate_preds.loc[
352     candidate_preds["model"] == CURRENT_MODEL,
353     ["window_start", "date", "store_id", "product_id", "category", "
        prediction"],
354 ]
355 current = current.merge(
356     original,
357     on=["window_start", "date", "store_id", "product_id", "category"],
358     how="left",
359 )
360
361 sum_cols = ["window_start", "date", *group_cols]
362 factor = (
363     current.groupby(sum_cols, dropna=False, as_index=False)
364     .agg(
365         pred_sum=("prediction", "sum"),
366         q1_sum=("q1_target_prediction", "sum"),
367     )
368     .reset_index(drop=True)
369 )
370 factor["calibration_factor"] = np.where(
371     factor["pred_sum"].abs().gt(1e-9),
372     factor["q1_sum"] / factor["pred_sum"],
373     1.0,
374 )
375 # Avoid making a diagnostic calibration dominate the validation through
    extreme ratios.
376 factor["calibration_factor"] = factor["calibration_factor"].clip(lower
    =0.5, upper=1.5)

```

```

377 out = current.merge(factor[sum_cols + ["calibration_factor"]], on=sum_cols
378 , how="left")
379 out["prediction"] = np.maximum(0.0, out["prediction"] * out["
380 calibration_factor"].fillna(1.0))
381 out["model"] = f"hybrid_current_calibrated_to_q1_{level}"
382 level_label = {"overall": "总量", "store": "门店", "product": "商品", "
383 category": "类别"}[level]
384 out["model_label"] = f"当前混合策略校准到问题一{level_label}预测"
385 out["strategy_rule"] = (
386     f"以后处理方式将当前混合策略在日期-{level_label}层面的预测总量按比例缩放到
387     问题一指数平滑聚合预测； "
388     "比例只使用两套模型预测值，不使用验证窗口真实销量"
389 )
390 out["threshold_rule"] = f"问题一{level_label}聚合预测后处理校准"
391 return out[candidate_preds.columns]
392
393 def add_calibrated_candidates(base: pd.DataFrame, candidates: pd.DataFrame) ->
394 pd.DataFrame:
395 calibrated = [
396     apply_q1_aggregate_calibration(base, candidates, level)
397     for level in ["overall", "store", "product", "category"]
398 ]
399 return pd.concat([candidates, *calibrated], ignore_index=True)
400
401 def aggregate_predictions(preds: pd.DataFrame, level: str) -> pd.DataFrame:
402 if level == "store_product":
403     return preds.copy()
404 if level == "store":
405     group_cols = ["date", "window_start", "model", "model_label", "store_id",
406                  "store_name"]
407 elif level == "product":
408     group_cols = [
409         "date",
410         "window_start",
411         "model",
412         "model_label",
413         "product_id",
414         "product_name",

```



```

411         "category",
412     ]
413     elif level == "category":
414         group_cols = ["date", "window_start", "model", "model_label", "category"]
415     else:
416         raise ValueError(level)
417     return (
418         preds.groupby(group_cols, dropna=False, as_index=False)
419         .agg(actual=("actual", "sum"), prediction=("prediction", "sum"))
420         .sort_values(group_cols)
421     )
422
423
424 def metric_row(df: pd.DataFrame) -> dict:
425     metrics = calculate_metrics(df["actual"], df["prediction"])
426     actual_sum = float(df["actual"].sum())
427     prediction_sum = float(df["prediction"].sum())
428     return {
429         **metrics,
430         "WAPE_pct": metrics["WAPE"] * 100 if pd.notna(metrics["WAPE"]) else np.
            nan,
431         "actual_sum": actual_sum,
432         "prediction_sum": prediction_sum,
433         "abs_error_sum": float((df["actual"] - df["prediction"]).abs().sum()),
434         "bias": prediction_sum - actual_sum,
435         "bias_pct_of_actual": (prediction_sum - actual_sum) / actual_sum
436         if actual_sum != 0
437         else np.nan,
438         "n": int(len(df)),
439     }
440
441
442 def build_metric_table(preds: pd.DataFrame) -> pd.DataFrame:
443     rows = []
444     for level in ["store_product", "store", "product", "category"]:
445         agg = aggregate_predictions(preds, level)
446         for (model, label), group in agg.groupby(["model", "model_label"],
            dropna=False):
447             original = preds[preds["model"] == model]

```

```

448         row = {
449             "validation_mode": "strict_recursive_7day",
450             "level": level,
451             "model": model,
452             "model_label": label,
453             "strategy_rule": original["strategy_rule"].dropna().iloc[0],
454             "threshold_rule": original["threshold_rule"].dropna().iloc[0],
455             "low_volume_selected_share": float(original["low_volume_selected
                "].mean()),
456             **metric_row(group),
457         }
458         rows.append(row)
459     out = pd.DataFrame(rows)
460     sp = out[out["level"] == "store_product"].set_index("model")
461     current_wape = float(sp.loc[CURRENT_MODEL, "WAPE_pct"])
462     q1_wape = float(sp.loc[Q1_MODEL, "WAPE_pct"])
463     ridge_wape = float(sp.loc[RIDGE_MODEL, "WAPE_pct"])
464     out["WAPE_pct_point_change_vs_current"] = out["WAPE_pct"] - current_wape
465     out["WAPE_pct_point_change_vs_q1"] = out["WAPE_pct"] - q1_wape
466     out["WAPE_pct_point_change_vs_ridge"] = out["WAPE_pct"] - ridge_wape
467     return out.sort_values(["level", "WAPE", "MAE", "RMSE"]).reset_index(drop=
        True)
468
469
470 def build_window_metric_table(preds: pd.DataFrame) -> pd.DataFrame:
471     rows = []
472     for (model, label, window_start), group in preds.groupby(
473         ["model", "model_label", "window_start"], dropna=False
474     ):
475         rows.append(
476             {
477                 "validation_mode": "strict_recursive_7day",
478                 "window_start": pd.Timestamp(window_start).date().isoformat(),
479                 "model": model,
480                 "model_label": label,
481                 **metric_row(group),
482             }
483         )
484     out = pd.DataFrame(rows)
485     current = out[out["model"] == CURRENT_MODEL][["window_start", "WAPE_pct"]

```

```

    ]].rename(
486         columns={"WAPE_pct": "current_window_WAPE_pct"}
487     )
488     out = out.merge(current, on="window_start", how="left")
489     out["WAPE_pct_point_change_vs_current_window"] = (
490         out["WAPE_pct"] - out["current_window_WAPE_pct"]
491     )
492     return out.sort_values(["window_start", "WAPE", "MAE", "RMSE"]).
        reset_index(drop=True)
493
494
495 def build_leave_one_window_selection(preds: pd.DataFrame) -> pd.DataFrame:
496     # Calibration rows are diagnostics, not competing low-volume decision
        rules.
497     candidate_pool = preds[~preds["model"].str.contains("calibrated", regex=
        False)].copy()
498     windows = sorted(pd.to_datetime(candidate_pool["window_start"]).
        drop_duplicates())
499     rows = []
500     for holdout in windows:
501         train = candidate_pool[candidate_pool["window_start"] != holdout]
502         holdout_df = candidate_pool[candidate_pool["window_start"] == holdout]
503         train_scores = []
504         for (model, label), group in train.groupby(["model", "model_label"],
            dropna=False):
505             train_scores.append({"model": model, "model_label": label, **
                metric_row(group)})
506         selected = pd.DataFrame(train_scores).sort_values(["WAPE", "MAE", "RMSE
            "]).iloc[0]
507         eval_models = [
508             (selected["model"], selected["model_label"], "
                selected_by_other_windows"),
509             (CURRENT_MODEL, "当前混合策略：低销量指数平滑-常规Ridge", "
                current_hybrid"),
510             (Q1_MODEL, "问题一简单指数平滑", "q1_baseline"),
511             (RIDGE_MODEL, "完整综合Ridge", "ridge_baseline"),
512         ]
513         for model, label, eval_type in eval_models:
514             group = holdout_df[holdout_df["model"] == model]
515             rows.append(

```

```

516         {
517             "holdout_window_start": holdout.date().isoformat(),
518             "selected_model_on_other_windows": selected["model"],
519             "selected_label_on_other_windows": selected["model_label"],
520             "selected_train_WAPE_pct": float(selected["WAPE_pct"]),
521             "eval_type": eval_type,
522             "eval_model": model,
523             "eval_label": label,
524             **metric_row(group),
525         }
526     )
527 out = pd.DataFrame(rows)
528 selected = out[out["eval_type"] == "selected_by_other_windows"][
529     ["holdout_window_start", "WAPE_pct"]
530 ].rename(columns={"WAPE_pct": "selected_holdout_WAPE_pct"})
531 current = out[out["eval_type"] == "current_hybrid"][
532     ["holdout_window_start", "WAPE_pct"]
533 ].rename(columns={"WAPE_pct": "current_holdout_WAPE_pct"})
534 out = out.merge(selected, on="holdout_window_start", how="left").merge(
535     current, on="holdout_window_start", how="left"
536 )
537 out["WAPE_pct_point_change_vs_current_holdout"] = (
538     out["WAPE_pct"] - out["current_holdout_WAPE_pct"]
539 )
540 return out.sort_values(["holdout_window_start", "eval_type"]).reset_index(
541     drop=True)
542
543 def load_calendar() -> pd.DataFrame:
544     path = ROOT / "data" / "processed" / "modeling_base_table.csv"
545     calendar = pd.read_csv(
546         path,
547         usecols=lambda col: col
548         in {"date", "weekday", "is_weekend", "is_holiday", "is_activity_day"},
549     )
550     calendar["date"] = pd.to_datetime(calendar["date"])
551     calendar = calendar.drop_duplicates("date")
552     for col in ["weekday", "is_weekend", "is_holiday", "is_activity_day"]:
553         calendar[col] = pd.to_numeric(calendar[col], errors="coerce").fillna(0)
554         .astype(int)

```

```

554     return calendar
555
556
557 def grouped_metrics(preds: pd.DataFrame, group_cols: list[str]) -> pd.
    DataFrame:
558     rows = []
559     for keys, group in preds.groupby(group_cols, dropna=False):
560         if not isinstance(keys, tuple):
561             keys = (keys,)
562             row = dict(zip(group_cols, keys))
563             row.update(metric_row(group))
564             rows.append(row)
565     if not rows:
566         return pd.DataFrame()
567     return pd.DataFrame(rows).sort_values(group_cols).reset_index(drop=True)
568
569
570 def build_error_attribution(
571     candidates: pd.DataFrame,
572 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, dict]:
573     hybrid = candidates[candidates["model"] == CURRENT_MODEL].copy()
574     hybrid = hybrid.merge(load_calendar(), on="date", how="left")
575     by_horizon = grouped_metrics(hybrid, ["horizon"])
576     by_class = grouped_metrics(hybrid, ["demand_class", "is_low_volume"])
577
578     segment_rows = []
579     segment_defs = [
580         ("工作日", "is_weekend=0 且 is_holiday=0", (hybrid["is_weekend"] == 0)
581             & (hybrid["is_holiday"] == 0)),
582         ("周末", "is_weekend=1", hybrid["is_weekend"] == 1),
583         ("非周末", "is_weekend=0", hybrid["is_weekend"] == 0),
584         ("活动日", "is_activity_day=1", hybrid["is_activity_day"] == 1),
585         ("非活动日", "is_activity_day=0", hybrid["is_activity_day"] == 0),
586         ("节假日", "is_holiday=1", hybrid["is_holiday"] == 1),
587         ("非节假日", "is_holiday=0", hybrid["is_holiday"] == 0),
588     ]
589     for label, definition, mask in segment_defs:
590         subset = hybrid.loc[mask].copy()
591         if subset.empty:
592             segment_rows.append(

```

```

592         {
593             "calendar_segment": label,
594             "definition": definition,
595             "MAE": np.nan,
596             "RMSE": np.nan,
597             "WAPE": np.nan,
598             "WAPE_pct": np.nan,
599             "actual_sum": 0.0,
600             "prediction_sum": 0.0,
601             "abs_error_sum": 0.0,
602             "bias": 0.0,
603             "bias_pct_of_actual": np.nan,
604             "n": 0,
605             "date_count": 0,
606         }
607     )
608     else:
609         row = {"calendar_segment": label, "definition": definition, **
610               metric_row(subset)}
611         row["date_count"] = int(subset["date"].nunique())
612         segment_rows.append(row)
613     by_calendar = pd.DataFrame(segment_rows)
614     save_csv(by_horizon, "q4_hybrid_error_by_horizon.csv")
615     save_csv(by_class, "q4_hybrid_error_by_demand_class.csv")
616     save_csv(by_calendar, "q4_hybrid_error_by_calendar_segment.csv")
617     low = hybrid[hybrid["is_low_volume"] == 1]
618     regular = hybrid[hybrid["is_low_volume"] == 0]
619     return by_horizon, by_class, by_calendar, {
620         "low_volume": metric_row(low) if not low.empty else {},
621         "regular": metric_row(regular) if not regular.empty else {},
622     }
623
624 def aggregate_7day_window(preds: pd.DataFrame, group_cols: list[str]) -> pd.
    DataFrame:
625     return (
626         preds.groupby(["window_start", *group_cols], dropna=False, as_index=
        False)
627         .agg(
628             actual_7day_sales=("actual", "sum"),

```

```

629         predicted_7day_sales=("prediction", "sum"),
630     )
631     .assign(
632         residual=lambda df: df["actual_7day_sales"] - df["
633             predicted_7day_sales"],
634         abs_residual=lambda df: df["residual"].abs(),
635     )
636
637
638 def interval_table_for_level(
639     validation: pd.DataFrame,
640     future: pd.DataFrame,
641     level: str,
642     group_cols: list[str],
643 ) -> pd.DataFrame:
644     val = aggregate_7day_window(validation, group_cols)
645     stats = (
646         val.groupby(group_cols, dropna=False)
647         .agg(
648             residual_q10=("residual", lambda x: float(np.quantile(x, 0.10))),
649             residual_median=("residual", "median"),
650             residual_q90=("residual", lambda x: float(np.quantile(x, 0.90))),
651             residual_mean=("residual", "mean"),
652             mean_abs_residual=("abs_residual", "mean"),
653             residual_std=("residual", "std"),
654             residual_count=("residual", "count"),
655             validation_actual_7day_mean=("actual_7day_sales", "mean"),
656             validation_prediction_7day_mean=("predicted_7day_sales", "mean"),
657         )
658         .reset_index()
659     )
660     fut = (
661         future.groupby(group_cols, dropna=False, as_index=False)["
662             predicted_sales"]
663         .sum()
664         .rename(columns={"predicted_sales": "point_forecast_7day_sales"})
665     )
666     out = fut.merge(stats, on=group_cols, how="left")
667     out["interval_method"] = "严格递推验证7日残差经验10%-90%分位数"

```

```

667 raw_lower = out["point_forecast_7day_sales"] + out["residual_q10"]
668 raw_upper = out["point_forecast_7day_sales"] + out["residual_q90"]
669 out["lower_80pct_empirical"] = np.maximum(
670     0.0, np.minimum(out["point_forecast_7day_sales"], raw_lower)
671 )
672 out["median_residual_adjusted_forecast"] = np.maximum(
673     0.0, out["point_forecast_7day_sales"] + out["residual_median"]
674 )
675 out["upper_80pct_empirical"] = np.maximum(out["point_forecast_7day_sales"],
676     raw_upper)
676 out.insert(0, "level", level)
677 return out.sort_values("point_forecast_7day_sales", ascending=False).
678     reset_index(drop=True)
679
680 def build_forecast_intervals(
681     candidates: pd.DataFrame,
682 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
683     future_path = OUTPUTS / "final_7day_forecast_hybrid_low_volume.csv"
684     if not future_path.exists():
685         raise FileNotFoundError("缺少 outputs/
686             final_7day_forecast_hybrid_low_volume.csv。")
686     future = pd.read_csv(future_path)
687     future["date"] = pd.to_datetime(future["date"])
688     future["predicted_sales"] = pd.to_numeric(future["predicted_sales"],
689         errors="coerce").fillna(0.0)
689
690     validation = candidates[candidates["model"] == CURRENT_MODEL].copy()
691     store = interval_table_for_level(
692         validation,
693         future,
694         "store",
695         ["store_id", "store_name"],
696     )
697     product = interval_table_for_level(
698         validation,
699         future,
700         "product",
701         ["product_id", "product_name", "category"],
702     )

```



```

703     category = interval_table_for_level(validation, future, "category", ["
        category"])
704     save_csv(store, "final_forecast_interval_by_store.csv")
705     save_csv(product, "final_forecast_interval_by_product.csv")
706     save_csv(category, "final_forecast_interval_by_category.csv")
707     return store, product, category
708
709
710 def plot_outputs(
711     metrics: pd.DataFrame,
712     by_horizon: pd.DataFrame,
713     store_interval: pd.DataFrame,
714 ) -> None:
715     setup_plot_style()
716
717     sp = metrics[metrics["level"] == "store_product"].head(12).sort_values("
        WAPE_pct")
718     fig, ax = plt.subplots(figsize=(10.5, 6.0))
719     labels = sp["model_label"].str.slice(0, 28)
720     colors = ["#2F6B55" if m == CURRENT_MODEL else "#6B7280" for m in sp["
        model"]]
721     ax.barh(labels, sp["WAPE_pct"], color=colors)
722     ax.set_xlabel("WAPE (%)")
723     ax.set_title("严格递推候选模型 WAPE 前 12")
724     for idx, row in enumerate(sp.itertuples()):
725         ax.text(row.WAPE_pct + 0.15, idx, f"{row.WAPE_pct:.2f}%", va="center",
            fontsize=8)
726     fig.tight_layout()
727     fig.savefig(FIGURES / "q4_candidate_model_wape.png", dpi=180)
728     plt.close(fig)
729
730     horizon = by_horizon.sort_values("horizon")
731     fig, ax = plt.subplots(figsize=(8.5, 4.8))
732     ax.bar(horizon["horizon"].astype(str), horizon["WAPE_pct"], color="#4C78A8
        ")
733     ax.set_xlabel("预测步长 horizon")
734     ax.set_ylabel("WAPE (%)")
735     ax.set_title("当前混合策略按 horizon 的严格递推误差")
736     for idx, row in enumerate(horizon.itertuples()):
737         ax.text(idx, row.WAPE_pct + 0.6, f"{row.WAPE_pct:.1f}%", ha="center",

```

```

        fontsize=8)
738 fig.tight_layout()
739 fig.savefig(FIGURES / "q4_hybrid_error_by_horizon.png", dpi=180)
740 plt.close(fig)
741
742 store = store_interval.sort_values("point_forecast_7day_sales", ascending=
    True)
743 fig, ax = plt.subplots(figsize=(10, 5.5))
744 x = store["point_forecast_7day_sales"].to_numpy()
745 lower = x - store["lower_80pct_empirical"].to_numpy()
746 upper = store["upper_80pct_empirical"].to_numpy() - x
747 ax.errorbar(
748     x,
749     store["store_name"],
750     xerr=np.vstack([lower, upper]),
751     fmt="o",
752     color="#2F6B55",
753     ecolor="#9CA3AF",
754     capsize=3,
755 )
756 ax.set_xlabel("未来 7 天预测销量")
757 ax.set_title("门店未来 7 天预测及经验残差区间")
758 fig.tight_layout()
759 fig.savefig(FIGURES / "final_forecast_interval_by_store.png", dpi=180)
760 plt.close(fig)
761
762 store_total_path = TABLES / "q4_hybrid_forecast_7day_total_by_store.csv"
763 category_total_path = TABLES / "q4_hybrid_forecast_7day_total_by_category.
    csv"
764 if store_total_path.exists():
765     store_total = pd.read_csv(store_total_path).sort_values(
766         "predicted_7day_sales", ascending=True
767     )
768     fig, ax = plt.subplots(figsize=(10, 5.4))
769     ax.barh(store_total["store_name"], store_total["predicted_7day_sales"],
        color="#2F6B55")
770     ax.set_xlabel("预测 7 日销量")
771     ax.set_title("低销量混合策略门店 7 日预测总量")
772     for idx, row in enumerate(store_total.itertuples()):
773         ax.text(

```

```

774         row.predicted_7day_sales + 3,
775         idx,
776         f"{row.predicted_7day_sales:.1f}",
777         va="center",
778         fontsize=8,
779     )
780     fig.tight_layout()
781     fig.savefig(FIGURES / "q4_hybrid_forecast_7day_total_by_store.png", dpi
782                 =180)
783     plt.close(fig)
784     if category_total_path.exists():
785         category_total = pd.read_csv(category_total_path).sort_values(
786             "predicted_7day_sales", ascending=True
787         )
788         fig, ax = plt.subplots(figsize=(10, 5.4))
789         ax.barh(category_total["category"], category_total["
790                 predicted_7day_sales"], color="#4C78A8")
791         ax.set_xlabel("预测 7 日销量")
792         ax.set_title("低销量混合策略类别 7 日预测总量")
793         for idx, row in enumerate(category_total.itertuples()):
794             ax.text(
795                 row.predicted_7day_sales + 3,
796                 idx,
797                 f"{row.predicted_7day_sales:.1f}",
798                 va="center",
799                 fontsize=8,
800             )
801         fig.tight_layout()
802         fig.savefig(FIGURES / "q4_hybrid_forecast_7day_total_by_category.png",
803                     dpi=180)
804         plt.close(fig)
805
806     def build_candidate_review(
807         metrics: pd.DataFrame,
808         window_metrics: pd.DataFrame,
809         leave_one: pd.DataFrame,
810     ) -> str:
811         sp = metrics[metrics["level"] == "store_product"].copy()
812         show_cols = [

```

```

811     "model_label",
812     "threshold_rule",
813     "MAE",
814     "RMSE",
815     "WAPE_pct",
816     "WAPE_pct_point_change_vs_current",
817     "actual_sum",
818     "prediction_sum",
819     "bias_pct_of_actual",
820     "low_volume_selected_share",
821 ]
822 top = sp[show_cols].head(15)
823 current = sp[sp["model"] == CURRENT_MODEL].iloc[0]
824 q1 = sp[sp["model"] == Q1_MODEL].iloc[0]
825 ridge = sp[sp["model"] == RIDGE_MODEL].iloc[0]
826 best = sp.iloc[0]
827 improvement = current["WAPE_pct"] - best["WAPE_pct"]
828 selected_mean = float(
829     leave_one.loc[leave_one["eval_type"] == "selected_by_other_windows", "
830         WAPE_pct"].mean()
831 )
832 current_holdout_mean = float(
833     leave_one.loc[leave_one["eval_type"] == "current_hybrid", "WAPE_pct"].
834     mean()
835 )
836 holdout_improvement = current_holdout_mean - selected_mean
837 replace_note = (
838     f"全样本最佳候选相对当前改善 {improvement:.3f} 个百分点，但留一窗口选模后
839     的平均改善为 "
840     f"{holdout_improvement:.3f} 个百分点，低于 0.3 个百分点阈值。因此本轮不自
841     动替换冻结主方案， "
842     "只将近 28 日均值兜底记为探索性候选。"
843 )
844 calibration = sp[sp["model"].str.contains("calibrated", regex=False)][
845     show_cols]
846 best_windows = window_metrics[window_metrics["model"] == best["model"]][
847     [
848         "window_start",
849         "model_label",
850         "WAPE_pct",

```

```

846         "WAPE_pct_point_change_vs_current_window",
847         "actual_sum",
848         "prediction_sum",
849     ]
850 ]
851 leave_cols = [
852     "holdout_window_start",
853     "selected_label_on_other_windows",
854     "eval_type",
855     "eval_label",
856     "WAPE_pct",
857     "WAPE_pct_point_change_vs_current_holdout",
858 ]
859
860 return f"""# 问题四候选模型有限优化复核
861
862 执行日期: {date.today().isoformat()}
863
864 ## 1. 复核目的
865
866 本轮只围绕当前严格 7 日递推口径做有限优化，重点检查两类问题：第一，低销量判定阈值
    和低销量兜底预测方法是否能稳定优于当前混合策略；第二，门店、商品、类别层级的比
    例校准是否能改善主粒度 WAPE。所有候选均基于已经生成的严格递推验证明细，不使用
    验证窗口真实销量构造未来特征，也不覆盖原最终预测表。
867
868 ## 2. 主粒度候选结果
869
870 {df_to_md(top, 20)}
871
872 完整结果见 `tables/q4_candidate_model_metrics.csv`，候选预测明细见 `tables/
    q4_candidate_model_predictions.csv`，对比图见 `figures/
    q4_candidate_model_wape.png`。
873
874 ## 3. 当前基准与替换判断
875
876 - 当前混合策略 WAPE={current['WAPE_pct']:.3f}%。
877 - 问题一简单指数平滑 WAPE={q1['WAPE_pct']:.3f}%。
878 - 完整综合 Ridge WAPE={ridge['WAPE_pct']:.3f}%。
879 - 本轮最佳候选为 `{best['model_label']}`，WAPE={best['WAPE_pct']:.3f}%，相对当
    前混合策略变化 {best['WAPE_pct_point_change_vs_current']:.3f} 个百分点。

```

```

880
881 结论: {replace_note}
882
883 ## 4. 窗口级稳健性
884
885 全样本最佳候选在各验证窗口的表现如下。负数表示该窗口优于当前混合策略:
886
887 {df_to_md(best_windows, 10)}
888
889 进一步用“留一窗口”方式检查模型选择偏差: 每次用另外 3 个窗口选择候选模型, 再在留
    出的 1 个窗口上评价。结果如下:
890
891 {df_to_md(leave_one[leave_cols], 20)}
892
893 留一窗口下, 选中候选的平均 WAPE={selected_mean:.3f}%, 当前混合策略平均 WAPE={
    current_holdout_mean:.3f}%, 平均改善 {holdout_improvement:.3f} 个百分点。该
    提升不足以把探索性候选写成新的主方案。
894
895 ## 5. 层级校准检查
896
897 层级校准只把当前混合策略的日期--层级预测总量按比例缩放到问题一指数平滑的同层级聚合
    预测; 该操作不使用验证期真实销量, 因此属于可复现后处理, 而不是利用真实答案校
    准。结果如下:
898
899 {df_to_md(calibration, 10)}
900
901 若层级校准不能在门店--商品主粒度上明显改善 WAPE, 则论文中只保留“检查过层级一致性和
    比例缩放, 未替换主方案”的结论。
902
903 ## 6. 论文写法建议
904
905 当前主方案仍应冻结为低销量指数平滑--常规 Ridge 混合策略。可以在模型评价中补充: 本
    轮尝试了不同低销量阈值、低销量均值兜底和层级校准, 虽然近 28 日均值兜底在全样本
    验证中表现更好, 但留一窗口检查未达到稳定替换阈值, 因此最终预测表继续使用 `
    outputs/final_7day_forecast_hybrid_low_volume.csv`。
906 ""
907
908
909 def build_error_report(
910     by_horizon: pd.DataFrame,

```

```

911     by_class: pd.DataFrame,
912     by_calendar: pd.DataFrame,
913     class_summary: dict,
914 ) -> str:
915     horizon_cols = ["horizon", "MAE", "RMSE", "WAPE_pct", "actual_sum", "
        prediction_sum", "n"]
916     class_cols = [
917         "demand_class",
918         "is_low_volume",
919         "MAE",
920         "RMSE",
921         "WAPE_pct",
922         "actual_sum",
923         "prediction_sum",
924         "n",
925     ]
926     calendar_cols = [
927         "calendar_segment",
928         "definition",
929         "MAE",
930         "RMSE",
931         "WAPE_pct",
932         "actual_sum",
933         "prediction_sum",
934         "date_count",
935     ]
936     worst_horizon = by_horizon.sort_values("WAPE_pct", ascending=False).iloc
        [0]
937     low_wape = class_summary.get("low_volume", {}).get("WAPE_pct", np.nan)
938     regular_wape = class_summary.get("regular", {}).get("WAPE_pct", np.nan)
939
940     return f"""# 问题四混合策略误差归因增强报告
941
942 执行日期: {date.today().isoformat()}
943
944 ## 1. 目的
945
946 本报告基于当前最终候选模型“低销量指数平滑--常规 Ridge 混合策略”的严格 7 日递推
    验证明细，补充按预测步长、需求类别和日历场景的误差归因。该报告服务于论文解释和
    答辩，不改变最终预测表。

```

```

947
948 ## 2. 按 horizon 的误差
949
950 {df_to_md(by_horizon[horizon_cols], 10)}
951
952 horizon={int(worst_horizon['horizon'])} 的 WAPE 最高, 为 {worst_horizon['
    WAPE_pct']:.2f}%。若后续步长误差升高, 可解释为递推预测中前序预测值进入后续
    lag/rolling 特征造成误差累积。
953
954 ## 3. 按低销量类别的误差
955
956 {df_to_md(by_class[class_cols], 20)}
957
958 低销量类别聚合 WAPE 约为 {low_wape:.2f}%, 常规类别聚合 WAPE 约为 {regular_wape
    :.2f}%。低销量组真实销量规模小, 少量绝对误差会被 WAPE 放大, 论文应同时引用
    MAE、RMSE 和真实销量合计。
959
960 ## 4. 按日历场景的误差
961
962 {df_to_md(by_calendar[calendar_cols], 20)}
963
964 周末、节假日和活动日样本可能重叠, 且验证窗口有限; 因此这些结果只能说明模型在不同场
    景上的误差表现, 不能作为因果判断。
965
966 ## 5. 输出文件
967
968 - `tables/q4_hybrid_error_by_horizon.csv`
969 - `tables/q4_hybrid_error_by_demand_class.csv`
970 - `tables/q4_hybrid_error_by_calendar_segment.csv`
971 - `figures/q4_hybrid_error_by_horizon.png`
972 """
973
974
975 def build_uncertainty_report(
976     store: pd.DataFrame,
977     product: pd.DataFrame,
978     category: pd.DataFrame,
979 ) -> str:
980     store_cols = [
981         "store_name",

```



```

982     "point_forecast_7day_sales",
983     "lower_80pct_empirical",
984     "median_residual_adjusted_forecast",
985     "upper_80pct_empirical",
986     "mean_abs_residual",
987     "residual_count",
988 ]
989 category_cols = [
990     "category",
991     "point_forecast_7day_sales",
992     "lower_80pct_empirical",
993     "median_residual_adjusted_forecast",
994     "upper_80pct_empirical",
995     "mean_abs_residual",
996     "residual_count",
997 ]
998 product_cols = [
999     "product_name",
1000     "category",
1001     "point_forecast_7day_sales",
1002     "lower_80pct_empirical",
1003     "upper_80pct_empirical",
1004     "mean_abs_residual",
1005 ]
1006
1007 return f"""# 未来 7 天预测不确定性报告
1008
1009 执行日期: {date.today().isoformat()}
1010
1011 ## 1. 区间构造方法
1012
1013 本报告不重新训练模型, 而是使用当前混合策略在 4 个严格 7 日递推验证窗口上的 7 日聚合残差构造经验区间。对每个门店、商品或类别, 先计算验证窗口中的
1014
1015 残差 = 真实7日销量 - 预测7日销量,
1016
1017 再取残差的 10% 和 90% 分位数加到未来 7 天点预测上, 形成经验 80% 残差区间。由于验证窗口只有 4 个, 该区间不是严格统计置信区间, 只用于反映历史递推误差下的预测不确定性。
1018

```

```

1019 ## 2. 门店级区间
1020
1021 {df_to_md(store[store_cols], 20)}
1022
1023 完整表见 `tables/final_forecast_interval_by_store.csv`, 图见 `figures/
    final_forecast_interval_by_store.png`。
1024
1025 ## 3. 类别级区间
1026
1027 {df_to_md(category[category_cols], 20)}
1028
1029 完整表见 `tables/final_forecast_interval_by_category.csv`。
1030
1031 ## 4. 商品级区间示例
1032
1033 下表按点预测销量从高到低展示前 15 个商品。完整表见 `tables/
    final_forecast_interval_by_product.csv`。
1034
1035 {df_to_md(product[product_cols], 15)}
1036
1037 ## 5. 使用边界
1038
1039 这些区间只来自历史验证残差, 不能覆盖未来天气、促销安排、库存缺货或突发大单等未观测
    变化。论文中适合表述为“基于严格递推验证残差的经验不确定性范围”, 不应写成保证
    真实销量落入区间的概率承诺。
1040 """
1041
1042
1043 def write_summary_json(
1044     metrics: pd.DataFrame,
1045     store_interval: pd.DataFrame,
1046     leave_one: pd.DataFrame,
1047 ) -> None:
1048     sp = metrics[metrics["level"] == "store_product"].copy()
1049     current = sp[sp["model"] == CURRENT_MODEL].iloc[0]
1050     best = sp.iloc[0]
1051     selected_mean = float(
1052         leave_one.loc[leave_one["eval_type"] == "selected_by_other_windows", "
            WAPE_pct"].mean()
1053     )

```

```

1054     current_holdout_mean = float(
1055         leave_one.loc[leave_one["eval_type"] == "current_hybrid", "WAPE_pct"].
1056         mean()
1057     )
1058     holdout_improvement = current_holdout_mean - selected_mean
1059     summary = {
1060         "validation_mode": "strict_recursive_7day",
1061         "current_model": CURRENT_MODEL,
1062         "current_wape_pct": float(current["WAPE_pct"]),
1063         "best_candidate_model": str(best["model"]),
1064         "best_candidate_label": str(best["model_label"]),
1065         "best_candidate_wape_pct": float(best["WAPE_pct"]),
1066         "best_minus_current_wape_pct_points": float(
1067             best["WAPE_pct"] - current["WAPE_pct"]
1068         ),
1069         "leave_one_selected_mean_wape_pct": selected_mean,
1070         "leave_one_current_mean_wape_pct": current_holdout_mean,
1071         "leave_one_improvement_pct_points": holdout_improvement,
1072         "replace_current": bool(
1073             current["WAPE_pct"] - best["WAPE_pct"] >= IMPROVEMENT_THRESHOLD_PCT
1074             and holdout_improvement >= IMPROVEMENT_THRESHOLD_PCT
1075             and best["model"] != CURRENT_MODEL
1076         ),
1077         "store_interval_rows": int(len(store_interval)),
1078     }
1079     (OUTPUTS / "q4_award_iteration_summary.json").write_text(
1080         json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
1081     )
1082
1083 def main() -> None:
1084     for path in [TABLES, OUTPUTS, FIGURES]:
1085         path.mkdir(parents=True, exist_ok=True)
1086
1087     base = load_strategy_base()
1088     candidates = build_candidate_predictions(base)
1089     candidates = add_calibrated_candidates(base, candidates)
1090     save_csv(candidates, "q4_candidate_model_predictions.csv")
1091
1092     metrics = build_metric_table(candidates)

```

```

1093 window_metrics = build_window_metric_table(candidates)
1094 leave_one = build_leave_one_window_selection(candidates)
1095 save_csv(metrics, "q4_candidate_model_metrics.csv")
1096 save_csv(window_metrics, "q4_candidate_window_metrics.csv")
1097 save_csv(leave_one, "q4_candidate_leave_one_window_selection.csv")
1098
1099 by_horizon, by_class, by_calendar, class_summary = build_error_attribution
        (candidates)
1100 store_interval, product_interval, category_interval =
        build_forecast_intervals(candidates)
1101 plot_outputs(metrics, by_horizon, store_interval)
1102
1103 (OUTPUTS / "q4_candidate_model_review.md").write_text(
1104     build_candidate_review(metrics, window_metrics, leave_one), encoding="
        utf-8"
1105 )
1106 (OUTPUTS / "q4_error_attribution_enhanced_report.md").write_text(
1107     build_error_report(by_horizon, by_class, by_calendar, class_summary),
        encoding="utf-8"
1108 )
1109 (OUTPUTS / "forecast_uncertainty_report.md").write_text(
1110     build_uncertainty_report(store_interval, product_interval,
        category_interval),
1111     encoding="utf-8",
1112 )
1113 write_summary_json(metrics, store_interval, leave_one)
1114
1115 sp = metrics[metrics["level"] == "store_product"].copy()
1116 current = sp[sp["model"] == CURRENT_MODEL].iloc[0]
1117 best = sp.iloc[0]
1118 improvement = current["WAPE_pct"] - best["WAPE_pct"]
1119 selected_mean = float(
1120     leave_one.loc[leave_one["eval_type"] == "selected_by_other_windows", "
        WAPE_pct"].mean()
1121 )
1122 current_holdout_mean = float(
1123     leave_one.loc[leave_one["eval_type"] == "current_hybrid", "WAPE_pct"].
        mean()
1124 )
1125 holdout_improvement = current_holdout_mean - selected_mean

```

```

1126 replace_note = (
1127     "全样本达到替换阈值但留一窗口未达阈值，作为探索候选但不替换冻结主方案"
1128     if improvement >= IMPROVEMENT_THRESHOLD_PCT and holdout_improvement <
        IMPROVEMENT_THRESHOLD_PCT
1129     else (
1130         "达到替换阈值，需人工复核候选预测表"
1131         if improvement >= IMPROVEMENT_THRESHOLD_PCT and best["model"] !=
            CURRENT_MODEL
1132         else "未达到 0.3 个百分点替换阈值，继续冻结当前混合策略"
1133     )
1134 )
1135 row = (
1136     f"| {date.today().isoformat()} | 问题四有限优化与不确定性补充 | "
1137     "processed: modeling_base_table.csv; 输入严格递推预测和混合策略预测；未修
        改 data/raw, 未覆盖原最终预测表 | "
1138     "低销量阈值候选、低销量均值兜底、层级比例校准、经验残差区间 | "
1139     "历史均值、零销量比例、近28日/商品/类别均值、问题一聚合预测校准、严格递推
        残差 | "
1140     f"当前混合策略 WAPE={current['WAPE_pct']:.2f}%；最佳候选={best['
        model_label']}] WAPE={best['WAPE_pct']:.2f}%； "
1141     f"相对当前全样本改善={improvement:.2f} 个百分点；留一窗口选模改善={
        holdout_improvement:.2f} 个百分点 | "
1142     f"{replace_note}；新增 horizon、低销量、日历场景误差和预测区间 | "
1143     "层级校准未使用验证窗口真实销量；预测区间为经验残差范围，不是严格置信区间
        | "
1144     "将候选复核、不确定性和误差归因写入论文与答辩材料 | "
1145 )
1146 prepend_result_log(row)
1147
1148 print(
1149     json.dumps(
1150         {
1151             "current_wape_pct": float(current["WAPE_pct"]),
1152             "best_model": str(best["model"]),
1153             "best_label": str(best["model_label"]),
1154             "best_wape_pct": float(best["WAPE_pct"]),
1155             "improvement_pct_points": float(improvement),
1156             "leave_one_selected_mean_wape_pct": float(selected_mean),
1157             "leave_one_current_mean_wape_pct": float(current_holdout_mean),
1158             "leave_one_improvement_pct_points": float(holdout_improvement),

```

```

1159         "replace_current": bool(
1160             improvement >= IMPROVEMENT_THRESHOLD_PCT
1161             and holdout_improvement >= IMPROVEMENT_THRESHOLD_PCT
1162             and best["model"] != CURRENT_MODEL
1163         ),
1164     },
1165     ensure_ascii=False,
1166     indent=2,
1167 )
1168 )
1169
1170
1171 if __name__ == "__main__":
1172     main()

```

A.4 AI 工具使用说明

AI 工具主要用于代码辅助、报告润色、审查提示、格式整理和 LaTeX 排版转换。字段解释、负销量口径、模型取舍、问题四验证口径和最终结论均需团队人工确认。AI 不替代团队对数据、模型和提交材料的最终判断。完整记录见 `outputs/ai_usage_record.md`。